Application Note

FESTO

VTEM with Beckhoff PLC through EtherCAT

This Application Note gives detailed procedure to integrate VTEM Motion Terminal with Beckhoff PLC through EtherCAT Communication Network. Also describe Library and Function block to control the VTEM Motion Terminal from Festo.

VTEM

100796

Title ..... VTEM with Beckhoff PLC through EtherCAT
Version 1.10
Document no. 100796
Original ..... en
Author ..... Festo

Last saved 08.01.2026

General information:

This document is intended for qualified, trained and instructed professionals. The data provided in this document are no guaranteed specifications, in particular with regard to functionality, condition or quality in the legal sense. The information in this document is intended only as simple indications for the implementation of a specific, hypothetical application, and in no way as a substitute for the operating instructions of the respective manufacturers or the design and testing of the respective application by the user. The respective operating instructions for Festo products can be found under www.festo.com. The user of this document must ensure that each function described herein works properly in his application. The user remains solely responsible for his or her own use of this document, even by studying this document and using the information mentioned therein. This also applies to any software (in source code and/or object code) that is made available to the user as an appendix to this document.

Rights of use of the software:

If software is made available to the user as an appendix to this document, the user is granted a simple and unlimited right of use hereto. This right also includes the processing and distribution of the software to third parties in edited or unedited form. The User is not permitted to use the name "Festo" to endorse or promote products derived from the Software without express prior written permission.

Software is provided to the user "as is". Festo does not assume any warranty or guarantee with regard to software. Festo's liability for damages of any kind arising from the use of the software is limited to intent and gross negligence.

Legal Notices:

©Festo SE & Co. KG, all rights reserved. A change in content and form is only permitted with the express written consent of Festo SE & Co. KG. Festo grants the user the right to reproduce this document in a form that remains unchanged in terms of content and form and to pass it on to third parties.

© (Festo SE & CO. KG, D-73726 Esslingen, 2026)
Internet: http://www.festo.com

Table of contents

1 Introduction 5

1.1 Software Used 5

1.2 Hardware Used 6

1.3 Topology Overview 6

2 Hardware Configuration 7

2.1 VTEM Motion Terminal Overview 7

2.2 CPX-FB37 Bus Node Overview 7

2.3 Overview of DIL Switch Setting of CPX-FB37 8

2.3.1 Setting Operating Mode and EtherCAT interface 8

2.3.2 Setting diagnostics mode, I/O bytes or starting of the bootloader 9

2.3.3 Setting the EtherCAT address 9

2.3.4 4-pin M12 socket (D-coded) Connection Details 9

3 Software Configuration 10

3.1 Beckhoff PLC Configuration in TwinCAT 3 10

3.1.1 Connect Beckhoff Controller Using TwinCAT 3 10

3.1.2 Creating a new project in TwinCAT 3 14

3.2 Configuring CPX-FB37 in TwinCAT 3 16

3.2.1 Adding ESI File of CPX-FB37 16

3.2.2 Adding CPX-FB37 module to I/O devices 18

3.3 I/O Address Configuration and Linking to Module I/O's in TwinCAT 3 22

3.3.1 PLC Variable Creation in TwinCAT 3 22

3.3.2 PLC Variables Link to Module I/Os 24

3.4 Adding Festo_VTEM_EC Library to TwinCAT 3 26

3.4.1 Importing Festo_VTEM_EC Library into TwinCAT 3 26

3.4.2 Adding Festo_VTEM_EC Library into TwinCAT Project 28

4 Festo_VTEM_EC Library Function Blocks Description 31

4.1 FB_ValveControl 31

4.1.1 FB_ValveControl Block Input Variable 32

4.1.2 FB_ValveControl Block Output Variable 33

4.1.3 FB_ValveControl Block In-Out Variables 33

4.2 FB_Upload_Single_Parameter 33

4.2.1 FB_Upload_Single_Parameter Block Input Variable 34

4.2.2 FB_Upload_Single_Parameter Block Output Variable 34

4.2.3 FB_Upload_Single_Parameter Block In-Out Variables 34

4.3 FB_Download_Single_Parameter 35

4.3.1 FB_Download_Single_Parameter Block Input Variable 35

4.3.2 FB_Download_Single_Parameter Block Output Variable 35

4.3.3 FB_Download_Single_Parameter Block In-Out Variables 36

4.4 FB_Save_Configuration ... 36
4.4.1 FB_Save_Configuration Block Input Variable ... 36
4.4.2 FB_Save_Configuration Block Output Variable ... 36
4.4.3 FB_Save_Configuration Block In-Out Variables ... 37
4.5 FB_ReadMalfunctionList ... 37
4.5.1 FB_ReadMalfunctionList Block Input Variable ... 37
4.5.2 FB_ReadMalfunctionList Block Output Variable ... 38
4.5.3 FB_ReadMalfunctionList Block In-Out Variable ... 38
4.6 Structure Data Types ... 39
4.6.1 stMalfunction ... 39
4.6.2 stTimeStamp ... 39
4.7 Enumeration Data Types ... 39
4.7.1 eResponseReadMalfunctionList ... 39
4.7.2 eResponseToValveModeSet ... 41
4.7.3 eSavingResponse ... 42
4.7.4 eTransferResponse ... 42
4.7.5 eValveState ... 43
5 Configuration of VTEM Function Blocks in TwinCAT 3 ... 44
5.1 Adding POU in Function Block Diagram Language ... 44
5.2 Configuring the Festo_VTEM_EC Function Block ... 45
6 Sample Code for Festo_VTEM_EC Library Function Blocks ... 50
6.1 Sample Code for Controlling the VTEM Motion Terminal Valve Using FB_ValveControl ... 50
6.2 Sample Code for Reading the Parameter Set value of the Valve from the VTEM Motion Terminal Valve using FB_Upload_Single_Parameter ... 51
6.3 Sample Code for Uploading Parameter from the VTEM Motion Terminal Valve using FB_Upload_Single_Parameter ... 52
6.4 Sample Code for Downloading Parameter to the VTEM Motion Terminal Valve using FB_Download_Single_Parameter ... 53
6.5 Sample Code for Save the VTEM Motion Terminal Valve Parameter Using FB_Save_Configuration ... 54
6.6 Sample Code for Read the Malfunction list of the VTEM Motion Terminal Valve Using FB_ReadMalfunctionList ... 54

1 Introduction

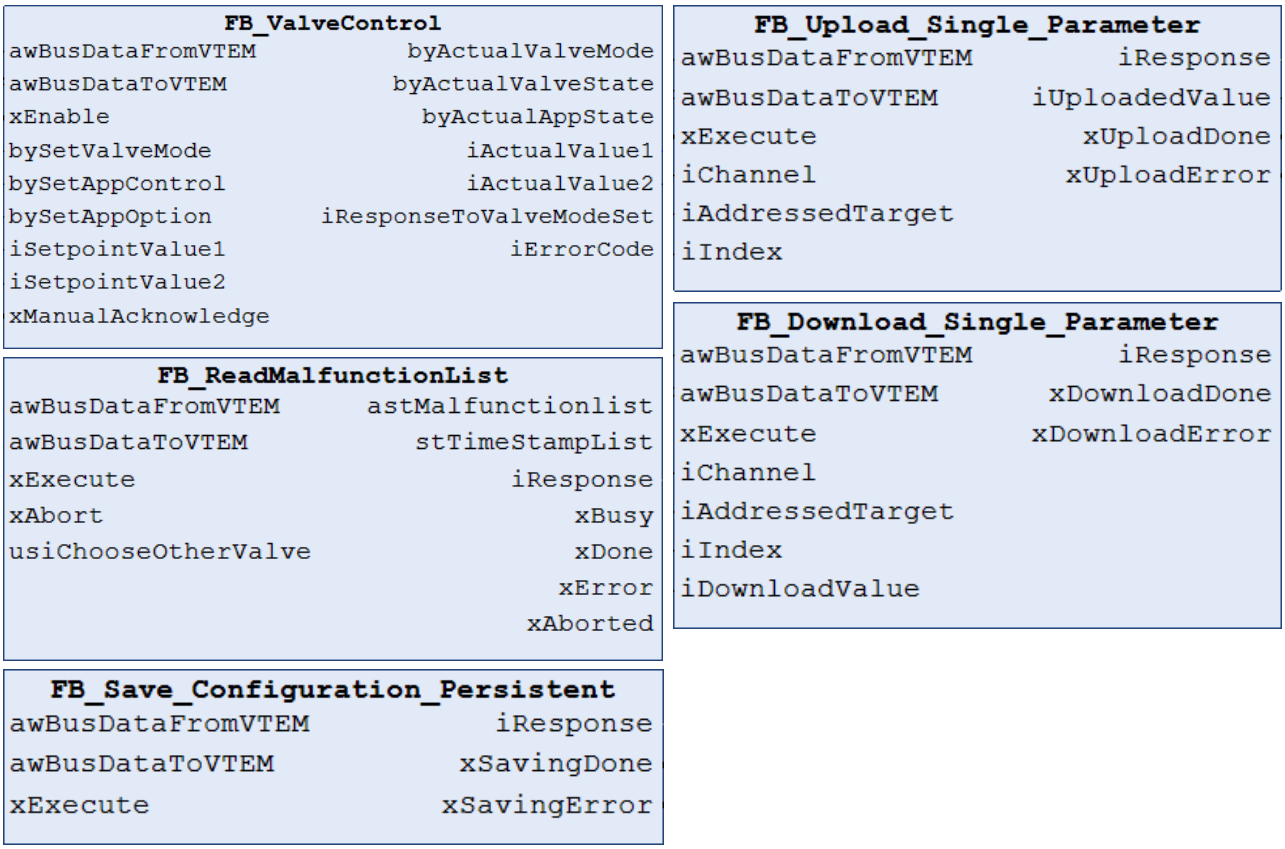
This application note demonstrates integration and controlling of the Festo VTEM motion terminal with Beckhoff PLC through EtherCAT.

This library contains five function blocks they are used to control, download, upload, save parameters to VTEM motion terminal & diagnose the error details of the motion terminal.

FB_ValveControl block is used to control the single valve in the motion terminal.

FB_Upload_Single_Parameter block is used to read the motion terminal valve parameter.
FB_Download_Single_Parameter block is used to write the motion terminal valve parameter.
FB_Save_Configuration block is used to save the motion terminal valve parameter.

FB_ReadMalfunctionList block is used to read the malfunction list of the VTEM motion terminal.



1.1 Software Used

Software Name	Software Version
Beckhoff TwinCAT 3	V3.1 Build 4024.71
Festo Automation Suite	V2.10.1.1
Festo-CPX-FB37 ESI File	V2211
Library (Festo_VTEM_EC)	V3.1.4024.710

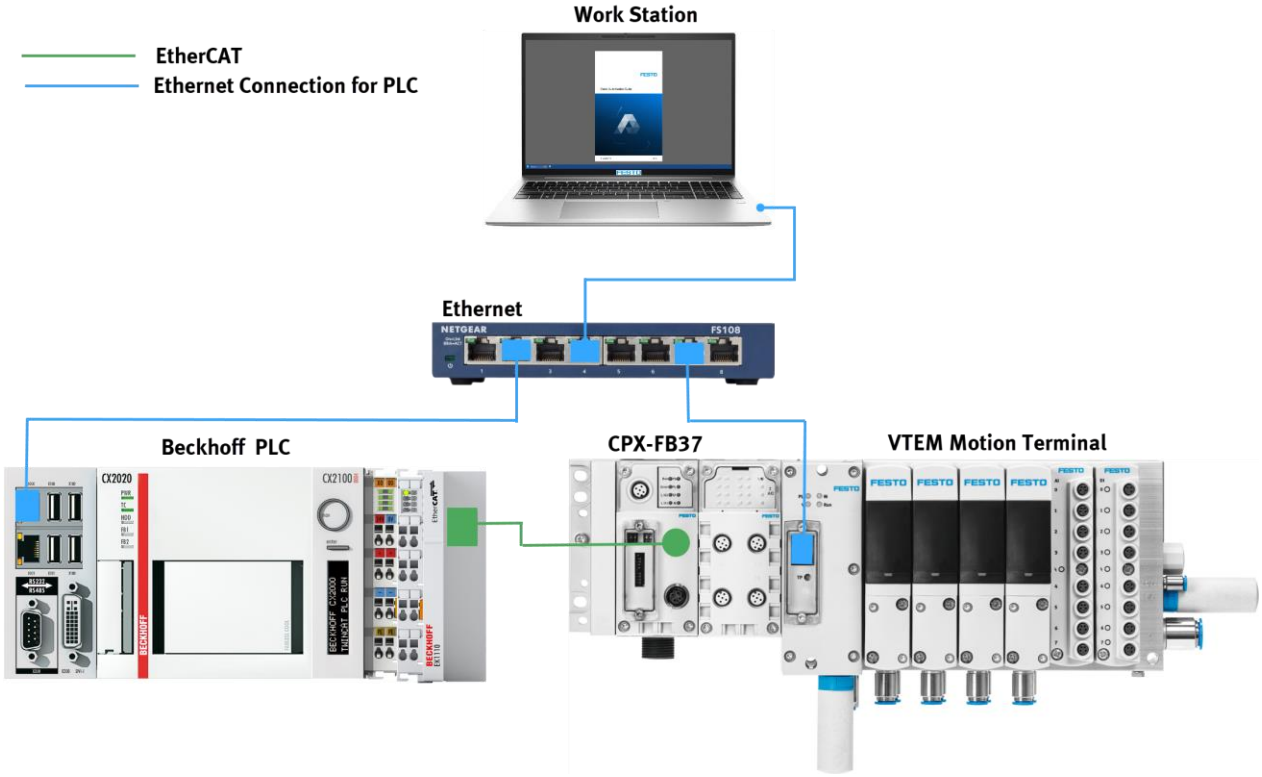
Table 1.1: Software used

1.2 Hardware Used

Device Name	Device Type	Version / Firmware
Beckhoff PLC	TwinCAT 3 CPU-CX2020	V3.86
Beckhoff Power Supply Unit	CX2100-0004	V2.5
Beckhoff EtherCAT Coupler	EK1110	V0017
Festo Bus Node	CPX-FB37	-
Festo Motion Terminal	VTEM	V4.26.6
M12 – RJ45 Ethernet Cable	NEBC-D12G4-ES-1-S-R3G4-ET	Version Free
RJ45 Ethernet Cable	Version Free	Version Free

Table 1.2: Hardware used

1.3 Topology Overview

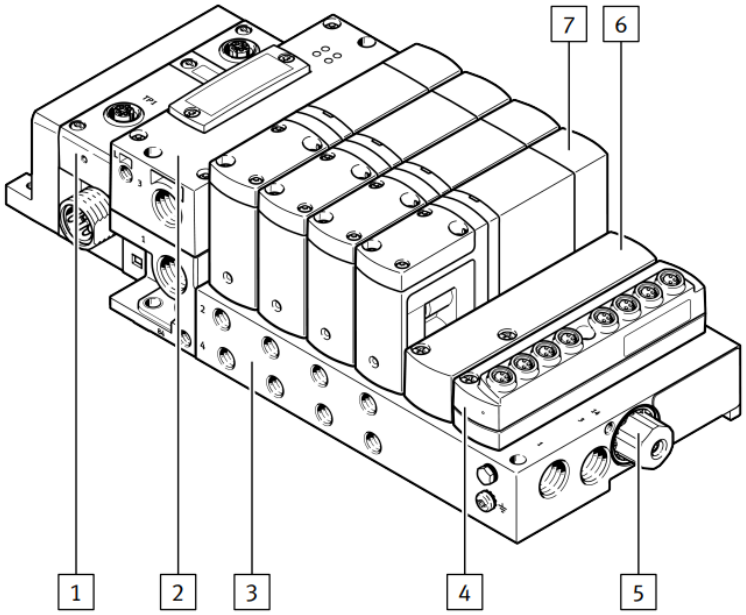


Note

- Above network configuration is an example network for this application note but depending on users network configuration, the architecture may change.

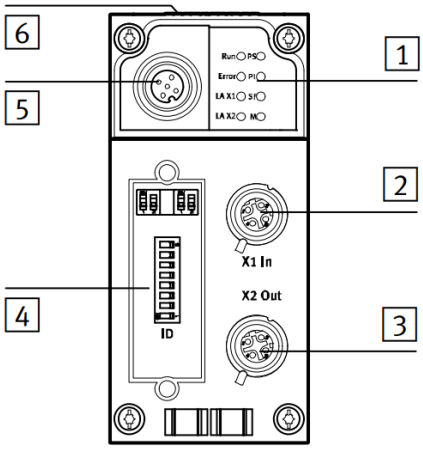
2 Hardware Configuration

2.1 VTEM Motion Terminal Overview



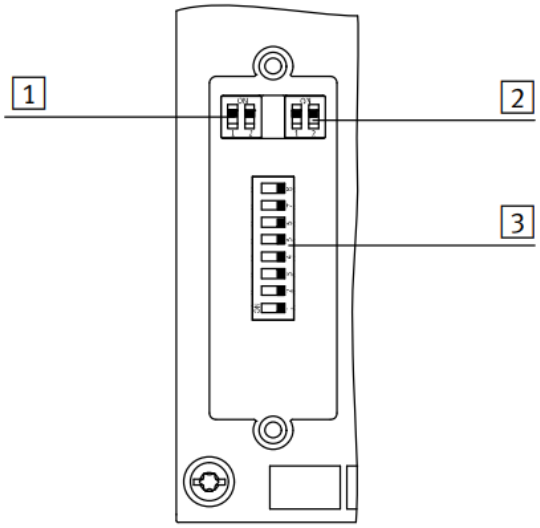
1 CPX terminal electronics side
2 Controller Motion Terminal CTMM-S1-C
3 Manifold rail
4 Input module CTMM-S1-A/D-... (optional)
5 Pilot pressure regulator
6 Cover plate VABB-P11-27-T (optional)
7 Valve VEVM-S1-27-...

2.2 CPX-FB37 Bus Node Overview



1 EtherCAT-specific network status LEDs and CPX-specific LEDs
2 Network connection 1 (input “X.1 In”)
3 Network connection 2 (output “X2 Out”)
4 Cover for DIL switches
5 Service interface for operator unit (CPX-MMI) and USB adapter for CPX-FMT
6 Rating plate

2.3 Overview of DIL Switch Setting of CPX-FB37

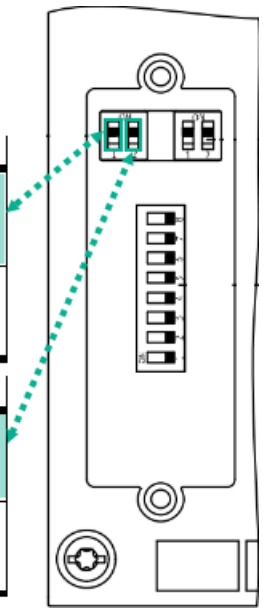


1 DIL switch 1: Operating mode, EtherCAT interface
2 DIL switch 2: Diagnostics mode, data field size, bootloader
3 DIL switch 3: EtherCAT address

2.3.1 Setting Operating Mode and EtherCAT interface

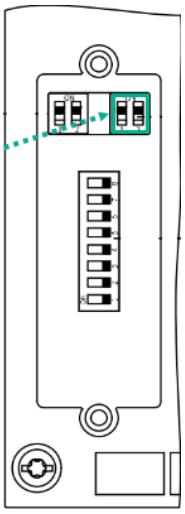
DIL switch 1	Setting	Operating mode
ON 1 2	DIL 1.1: OFF (Factory setting)	**Remote I/O** All functions of the CPX terminal are controlled directly by the EtherCAT I/O controller or a higher-level PLC.
ON 1 2	DIL 1.1: ON	**Remote controller** A CPX-FEC or CPX-CEC integrated into the terminal controls all functions

DIL switch 1	Setting	Function
ON 1 2	DIL 1.2: OFF (Factory setting)	**Modular device profile (MDP)**
ON 1 2	DIL 1.2: ON	**Fixes I/O size (64 byte I/A), compatible with the bus node CPX-FB38**

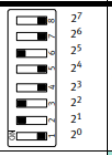
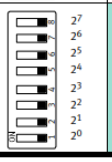


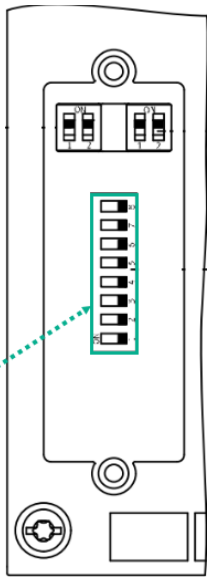
2.3.2 Setting diagnostics mode, I/O bytes or starting of the bootloader

DIL switch 2	Setting	Remote I/O	Remote controller
	DIL 2.1: OFF DIL 2.2: OFF (Factory setting)	I/O diagnostics interface and status bits **switched off**	8 byte I/8 byte O for communication of the bus node with the CPX-FEC or CPX-CEC
	DIL 2.1: OFF DIL 2.2: ON	**Status bits** are switched on + 16 I bits (only 8 bits used)	Reserved
	DIL 2.1: ON DIL 2.2: OFF	**I/O diagnostics interface** is switched on + 16 I/O bits	16 byte I/16 byte O for communication of the bus node with the CPX-FEC or CPX-CEC
	DIL 2.1: ON DIL 2.2: ON	Bootloader (Restore the firmware in case of loss of function)	Bootloader (Restore the firmware in case of loss of function)



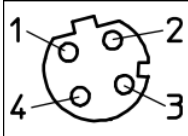
2.3.3 Setting the EtherCAT address

DIL switch 3	Function
	Binary-coded input of the EtherCAT address[1)] Example on left in figure: EtherCAT address $= 2^1 + 2^2 + 2^5$ $= 2 + 4 + 32 = 38$
	Binary-coded input of the EtherCAT address Example on left in figure (all switch elements are OFF): EtherCAT address = 0, the programmed EtherCAT address in the EEPROM is used (factory setting).



To avoid setting the CPX-FB37 module's IP address via DIL switches, ensure all switches from DIL 3.1 to 3.8 are off before configuring the IP address with the Festo Automation Suite software.

2.3.4 4-pin M12 socket (D-coded) Connection Details

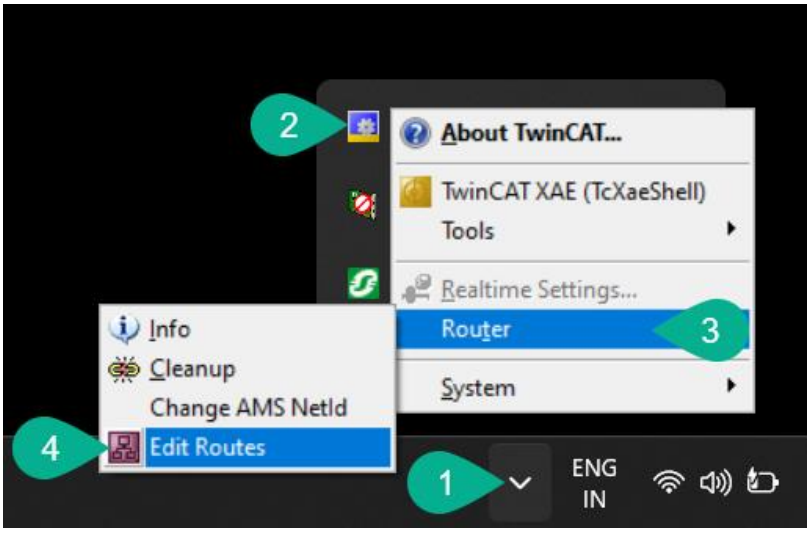
M12 socket, D-coded	Pin	Signal	Explanation
	1	TD+	Transmitted data (Transmit Data) +
	2	RD+	Received data (Receive Data) +
	3	TD–	Transmitted data –
	4	RD–	Received data –
	Housing	FE	Screening/functional earth

3 Software Configuration

3.1 Beckhoff PLC Configuration in TwinCAT 3

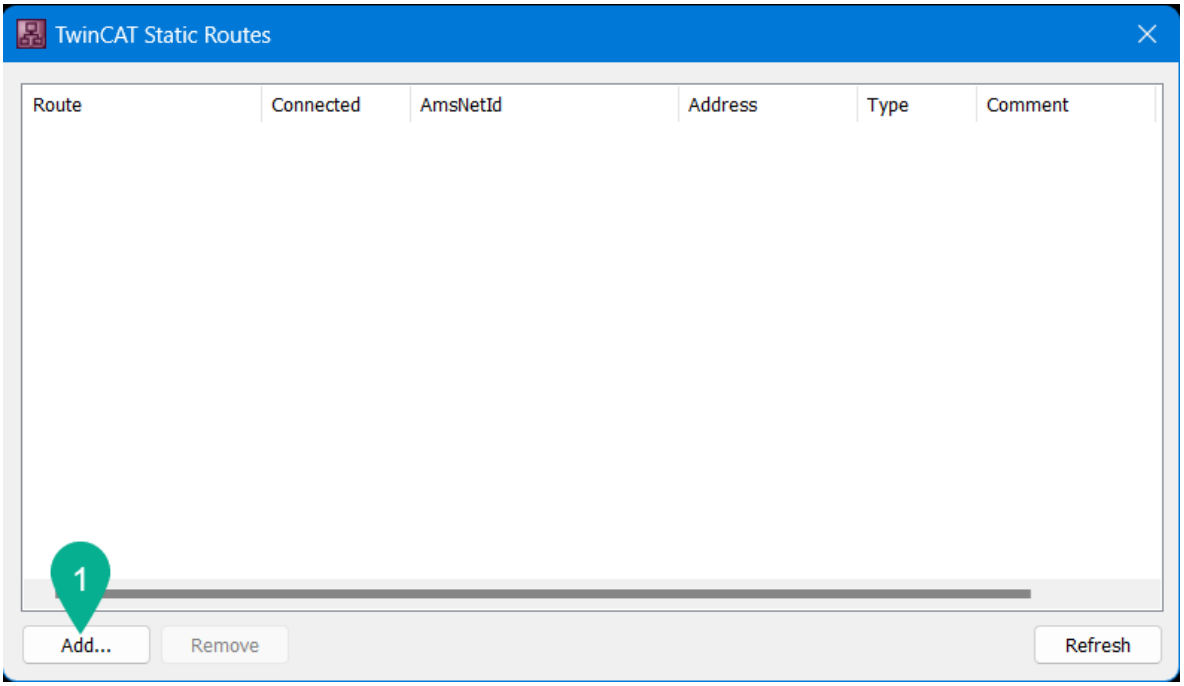
3.1.1 Connect Beckhoff Controller Using TwinCAT 3

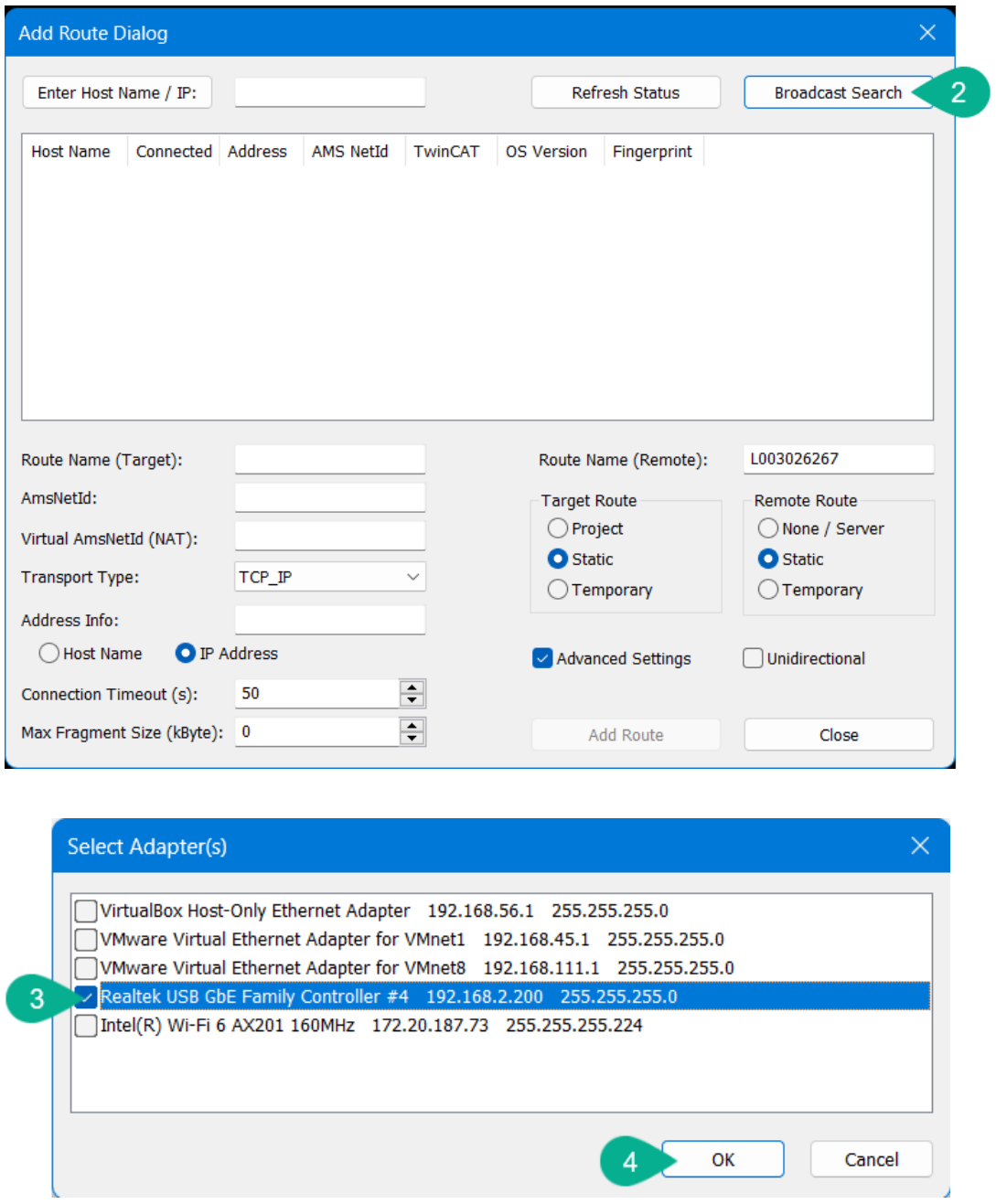
Step 1



1. Click on Taskbar hidden icon arrow from desktop window.
2. Click on **TwinCAT Config Mode** from taskbar hidden icons.
3. Click on **Router** from TwinCAT Config Mode popup window.
4. Click on **Edit Routes** to open the TwinCAT Static Routes Window.

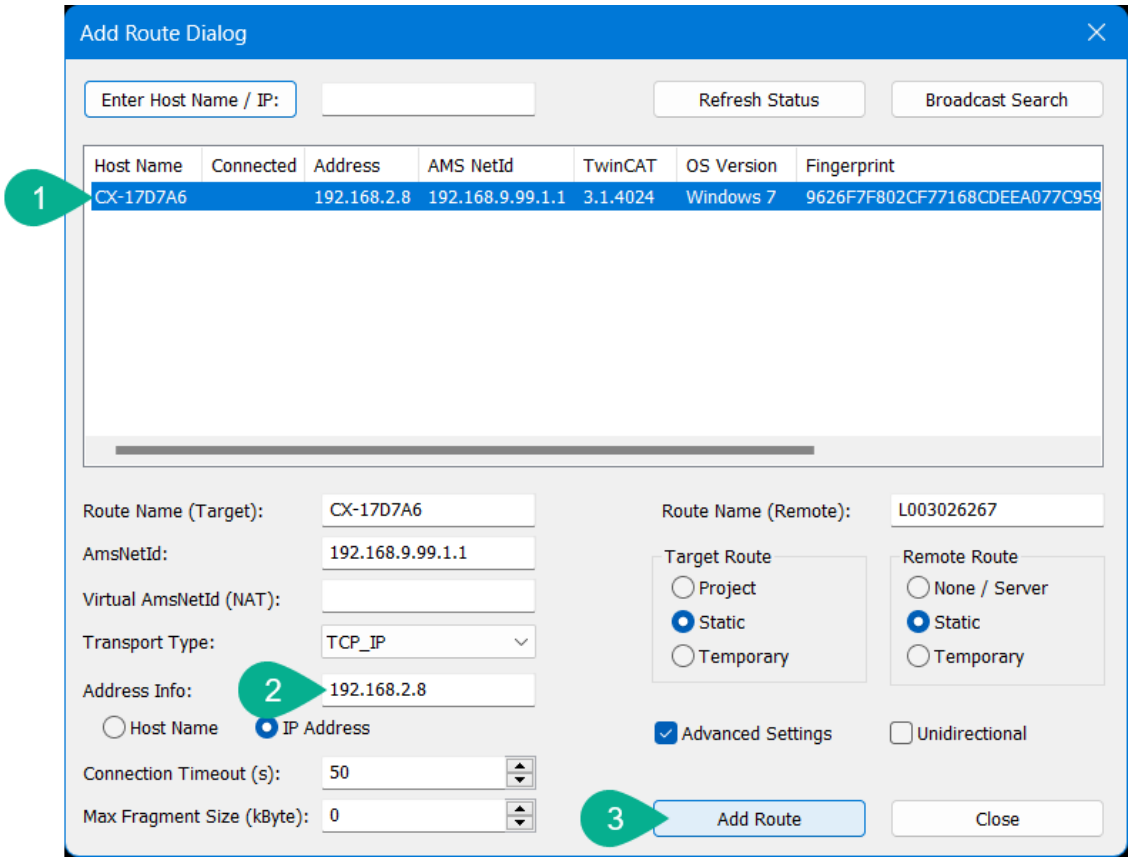
Step 2





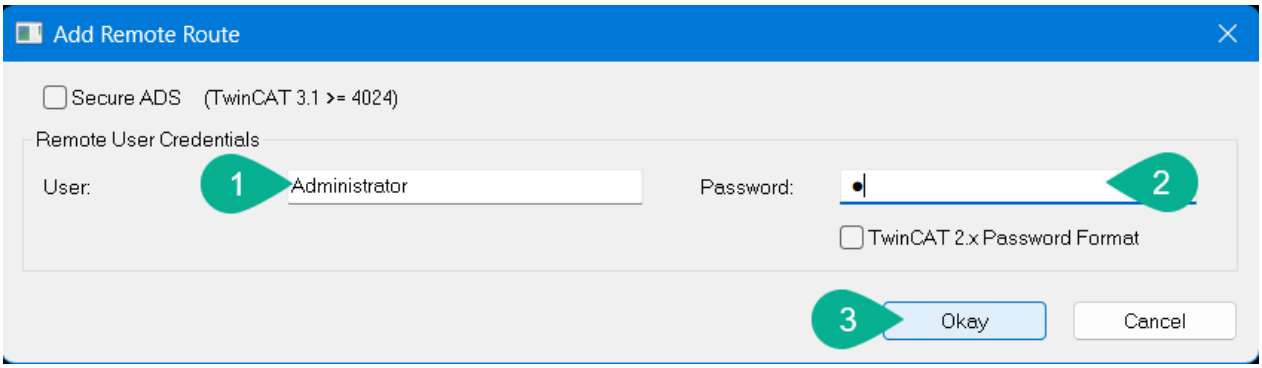
1. Click on **Add** from TwinCAT Static Route window.
2. Click on **Broadcast Search** after confirm Route Settings from Add Route Dialog window.
3. Select the correct adapter from the Adapter window.
4. Click **OK** to confirm the search in **Select Adapter(s)** window.

Step 3



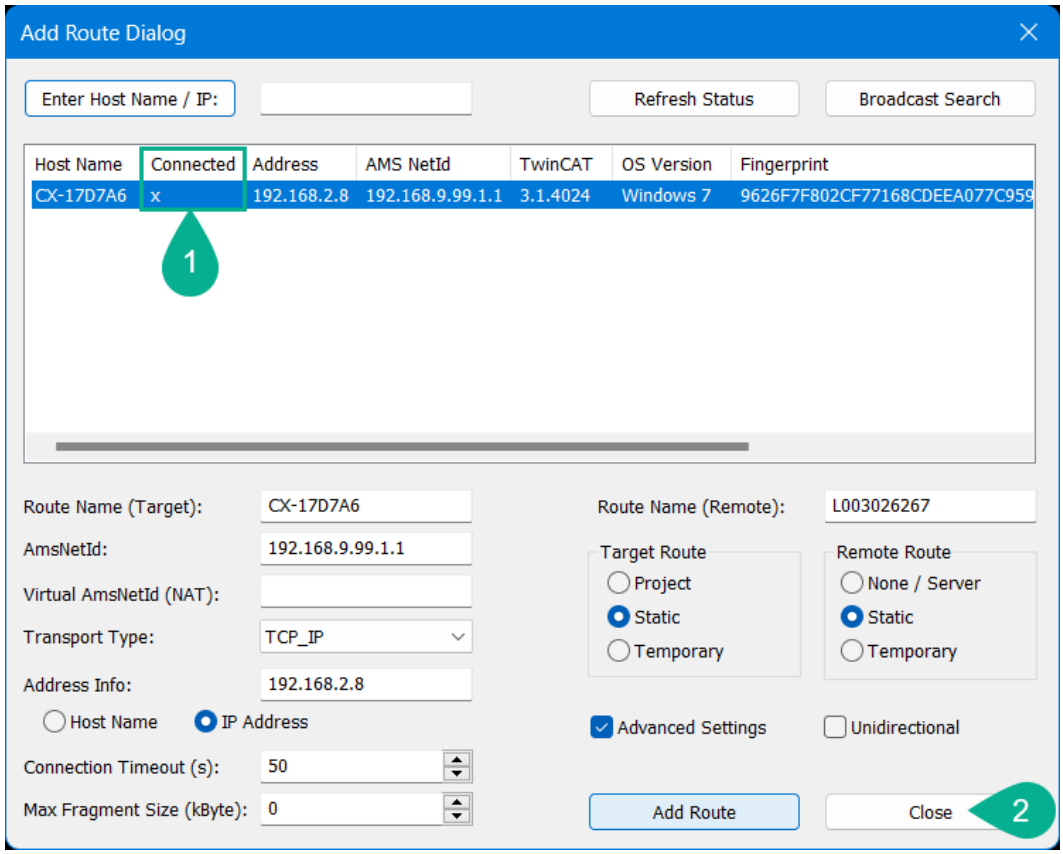
1. Select the **PLC Route Connection** from Add Route Dialog window.
2. Check the **IP Address of controller** on Address Info field.
3. Click on **Add Route** from Add Route Dialog window.

Step 4



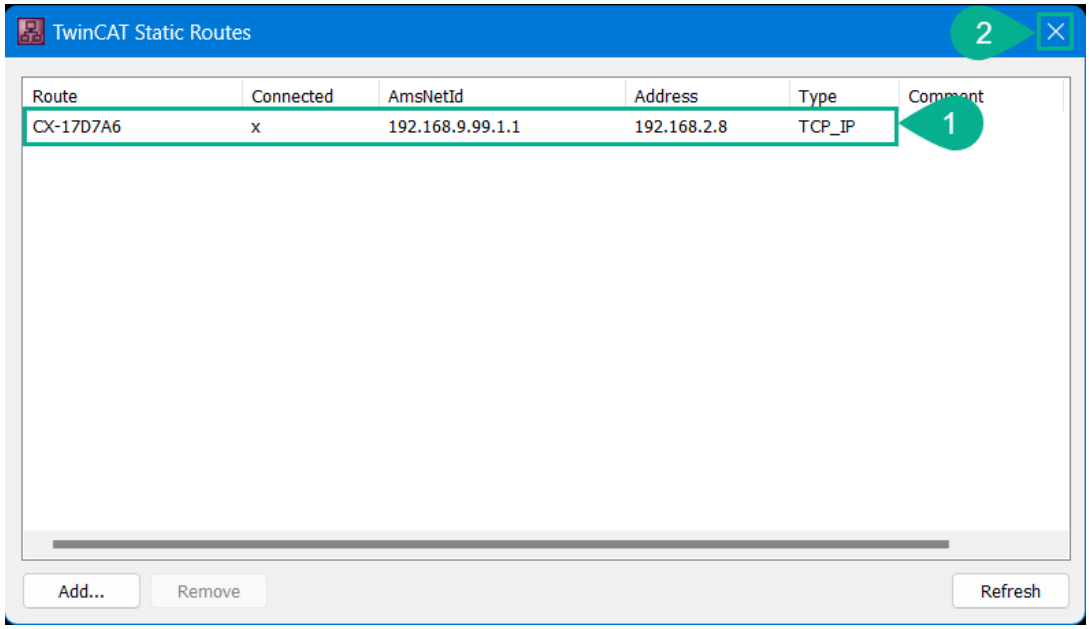
1. Enter the **User** name in User field.
2. Enter the **Password** in Password field.
3. Click on **Okay** to add the Remote Route**.**

Step 5



1. Check the Controller connect status (**x**) under Connected filed in Add Route Dialog window.
2. Click on **Close** from Add Route Dialog window**.**

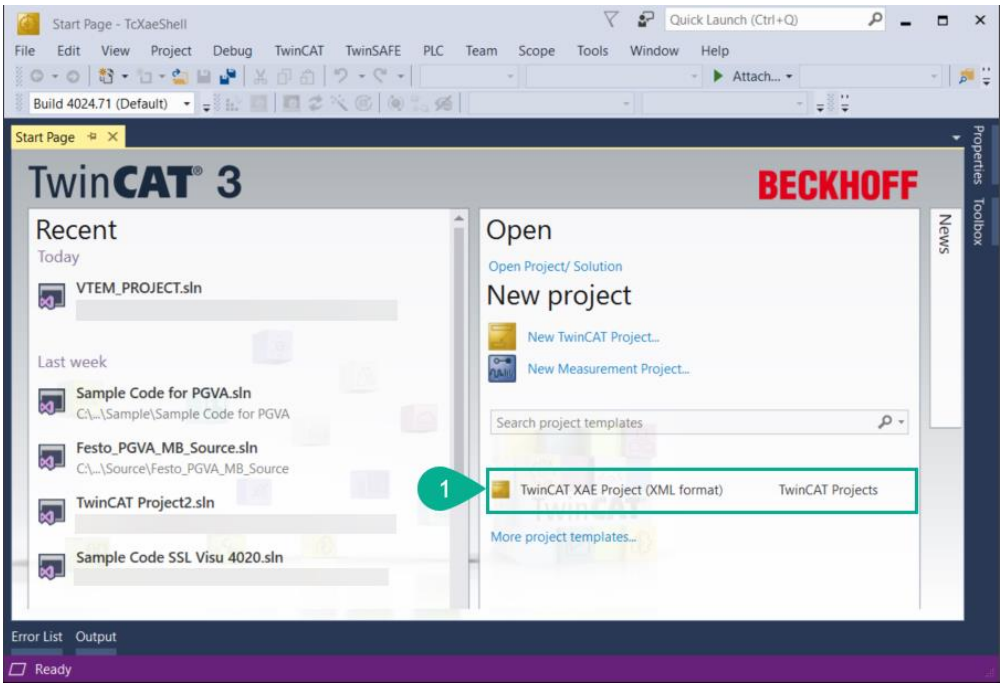
Step 6



1. Verify the available Route from TwinCAT Static Routes window.
2. Click on **Close** from TwinCAT Static Routes window**.**

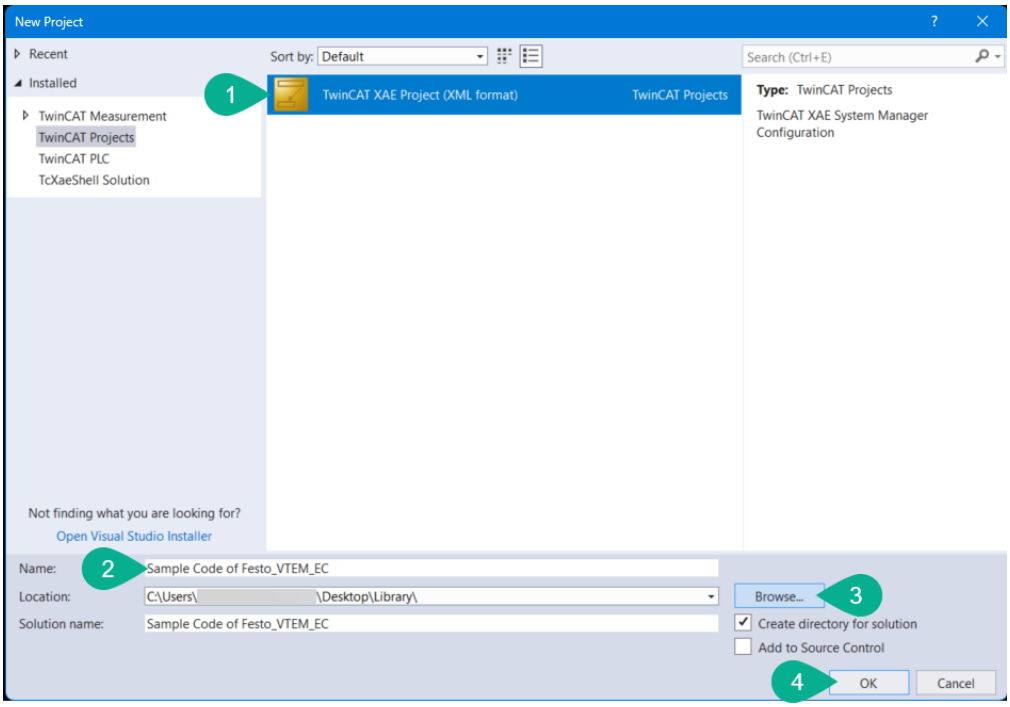
3.1.2 Creating a new project in TwinCAT 3

Step 1



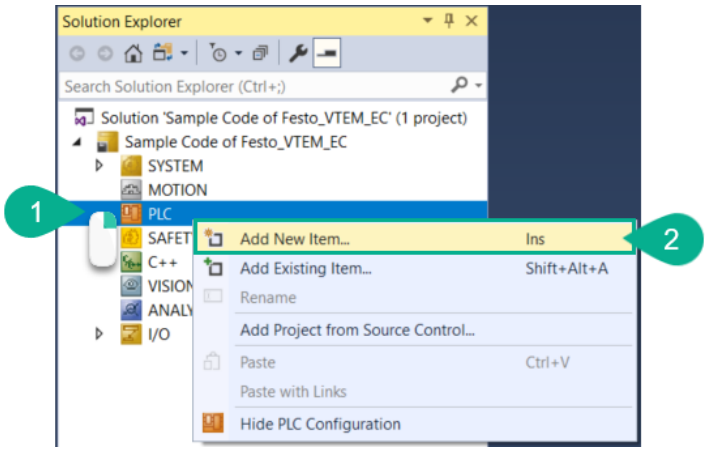
1. Click on **TwinCAT XAE Project** (**XML Format**) to create a new project.

Step 2



1. Select **TwinCAT XAE Project** (**XML Format**) from New Project tab.
2. Type the **Project name** on the Name field.
3. Select the **Location** where the project has to be saved.
4. Click **OK** to create project.

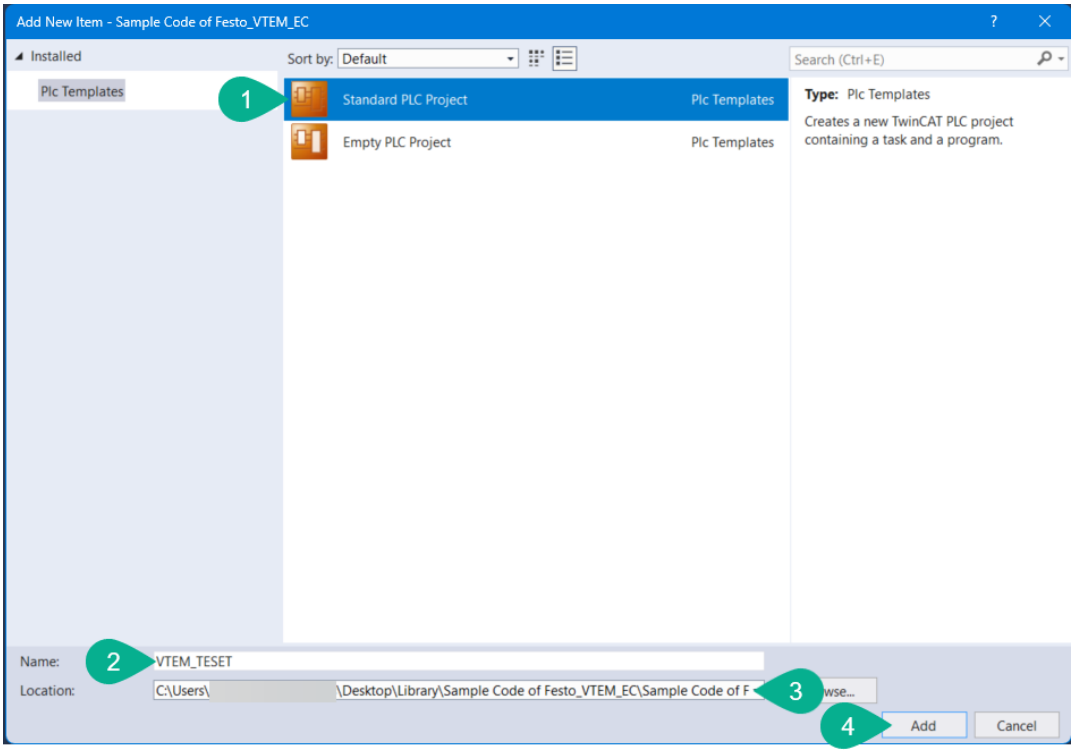
Step 3



1. Right Click **PLC** from Solution Explorer.
2. Click on **Add New item** to add the PLC.

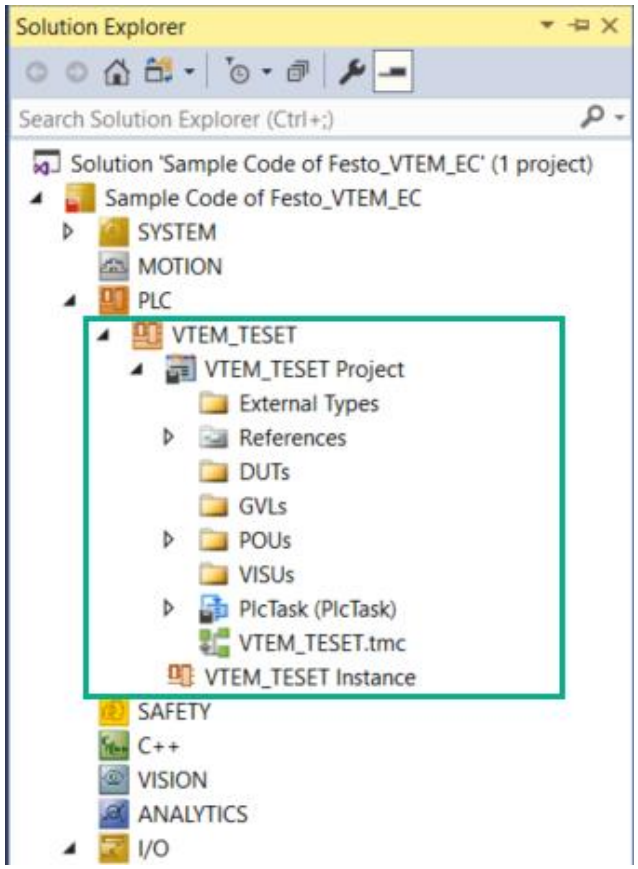
The Add New item tab will be displayed as shown below Step 4

Step 4



1. Select **Standard PLC Project** from Add New item tab.
2. Type the **PLC Project name** on the Name field.
3. Not required to change **location**, since it is set automatically.
4. Click on **Add** to add PLC Project.

The PLC Project will be added to the Solution Explorer as shown below.



3.2 Configuring CPX-FB37 in TwinCAT 3

3.2.1 Adding ESI File of CPX-FB37

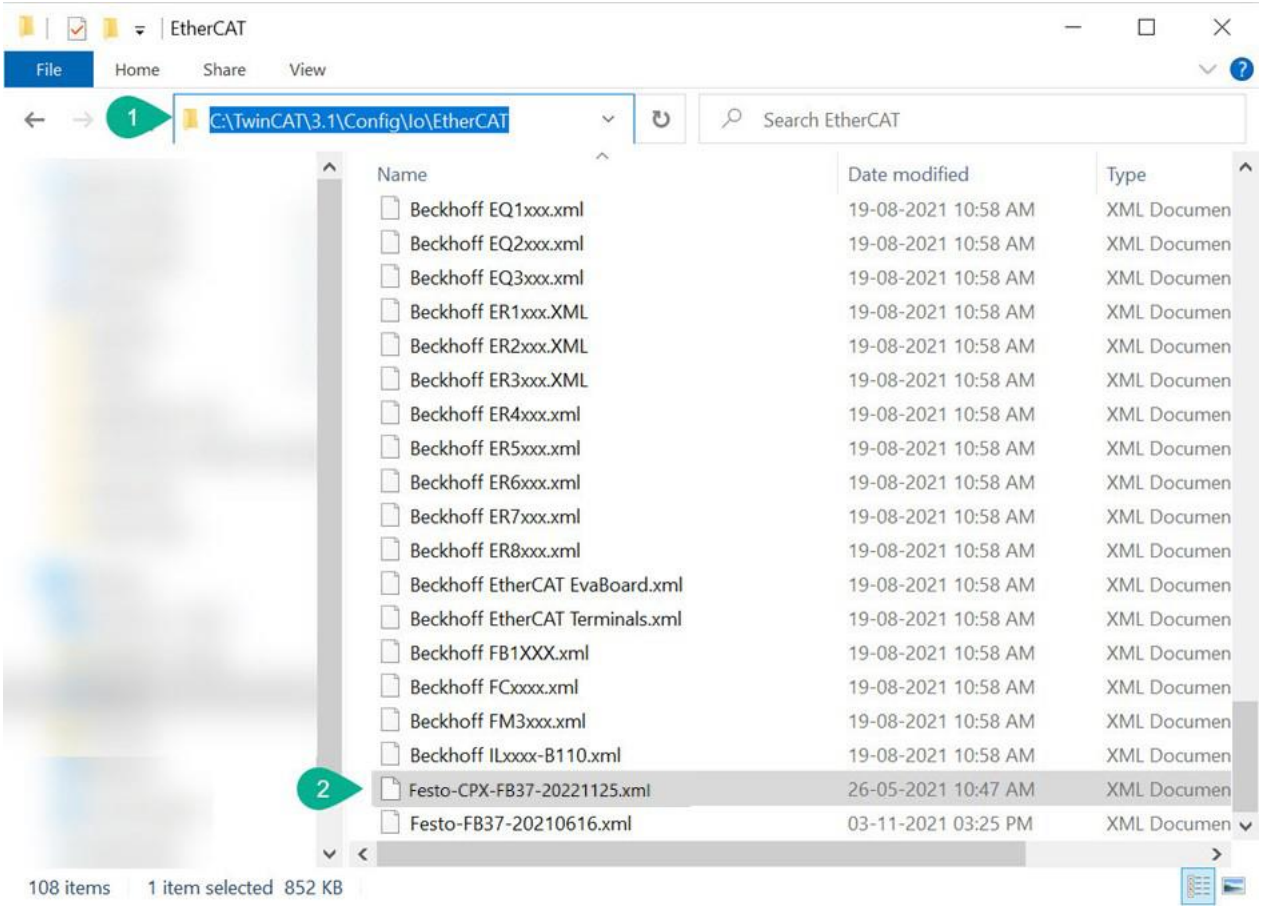
Step 1

Download the latest EtherCAT ESI File for **CPX-FB37** module from the Festo Support Portal.

Step 2

Copy the EtherCAT ESI File in the following link in the system.

1. **C:\TwinCAT\3.1\Config\Io\EtherCAT**

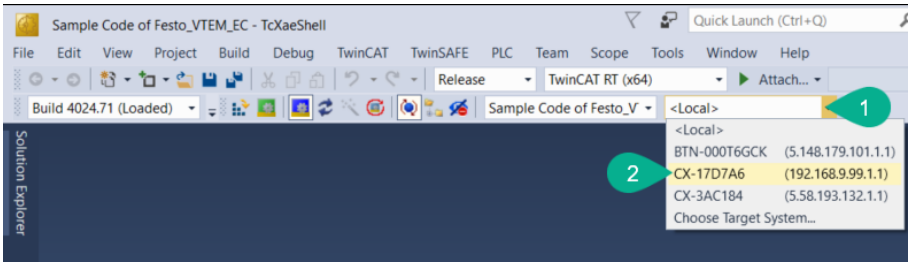


1. Copy the above EtherCAT ESI file path and paste in to path tab to open the TwinCAT 3 ESI file location.
2. Copy the EtherCAT ESI file from Downloads and paste it to **C:\TwinCAT\3.1\Config\Io\EtherCAT** folder.

3.2.2 Adding CPX-FB37 module to I/O devices

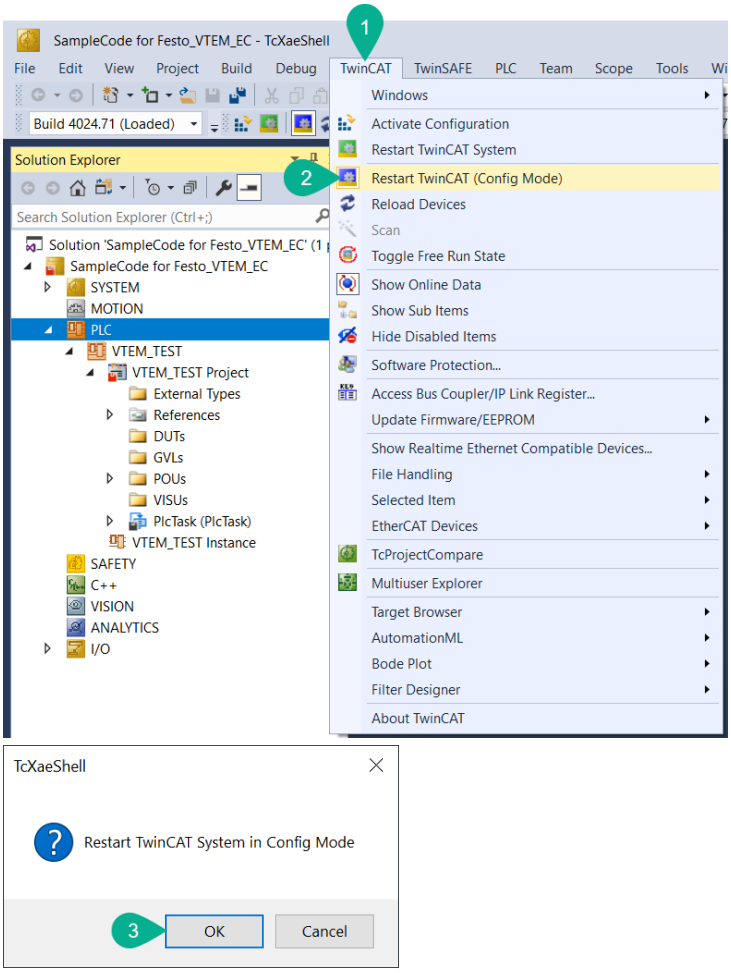
Choose Target System

Step 1



1. Click on drop down icon in Choose Target system field.
2. Choose **CX-17D7A6 Route** from Choose Target system drop down.

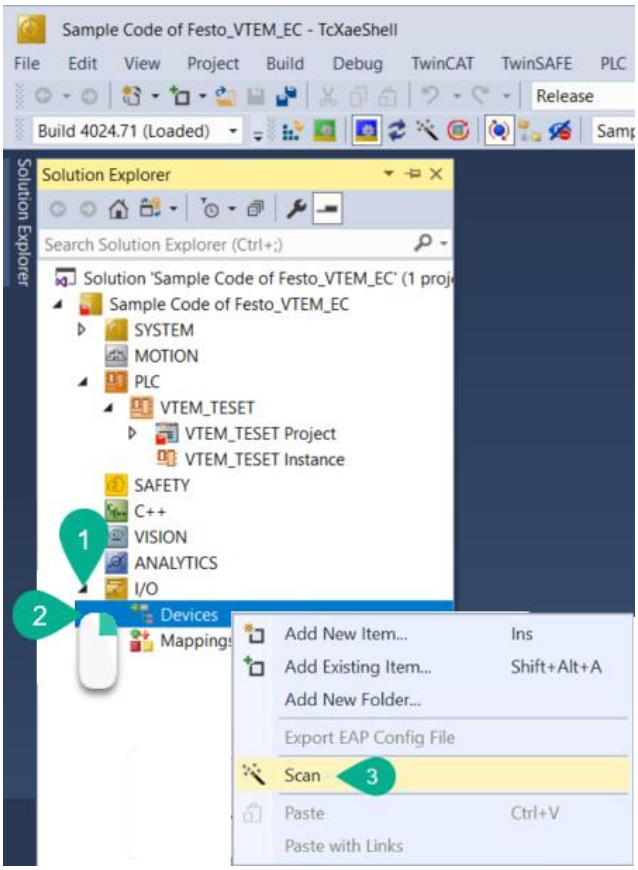
Step 2

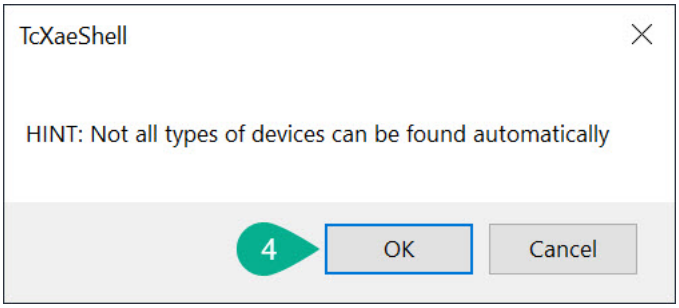


1. Click on **TwinCAT** menu from the top menu bar.
2. Click on **Restart TwinCAT (Config Mode)** from the TwinCAT menu.
3. Click on **OK** button to start the system in Config Mode.

Scanning I/O devices

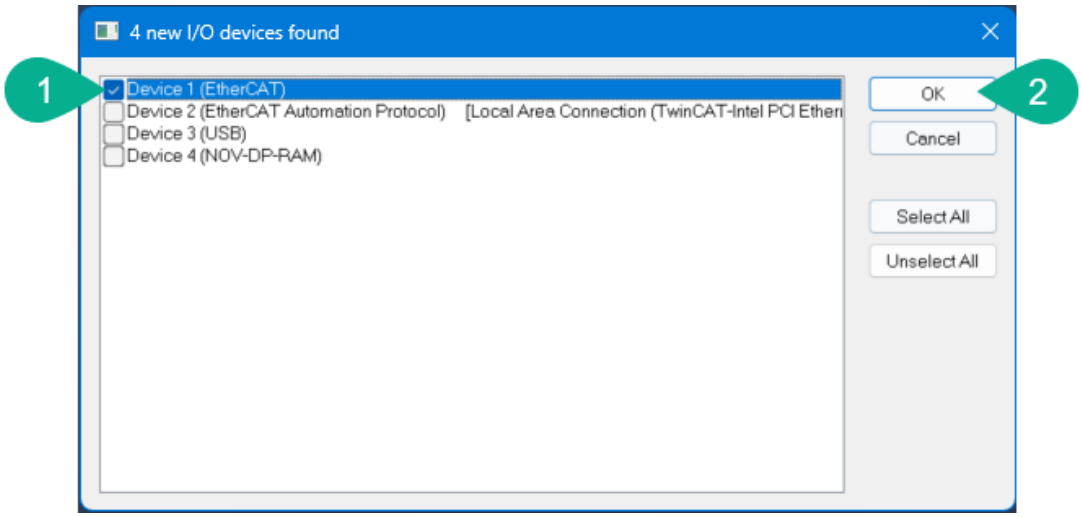
Step 1



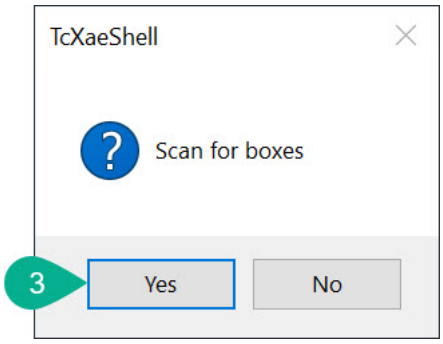


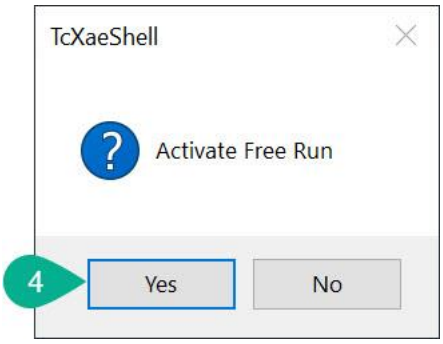
1. Expand the **I/O** from Solution Explorer.
2. Right click on **Device**s from I/O drop down menu.
3. Click on **Scan** to scan the I/O Devices.
4. Click **OK** button to start scanning for the I/O devices connected in the network.

Step 2



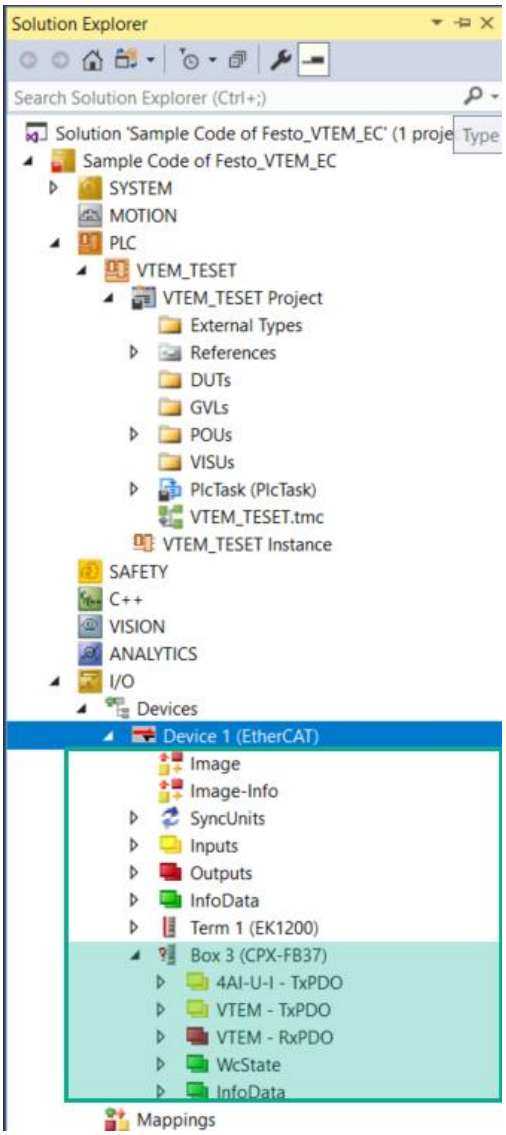
1. Select the **Device 1 (EtherCAT)** from I/O devices found window.
2. Click **OK** to add the I/O devices to project in new I/O devices found window.





3. Click **Yes** to Scan for Boxes which are connected to the EtherCAT Master.
4. After the Boxes are scanned, click **Yes** to activate Free Run.

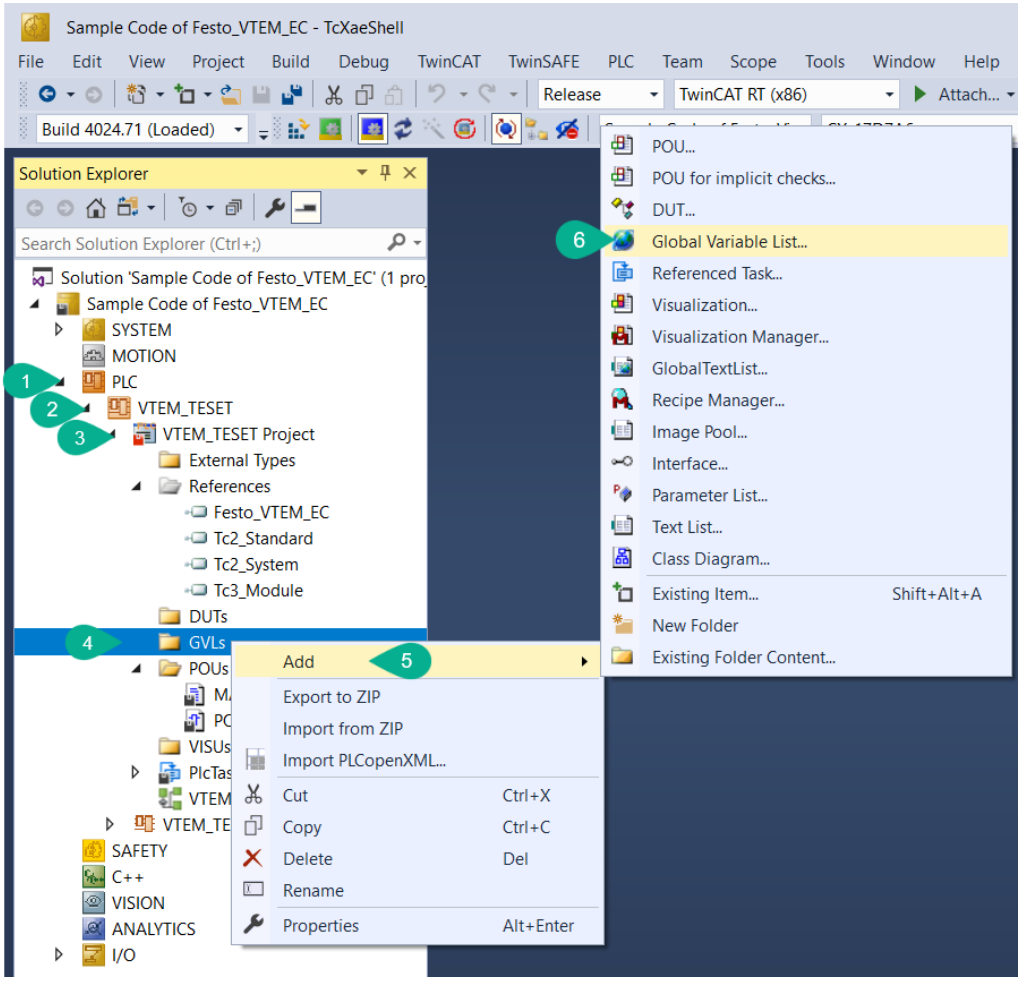
The devices connected in Network will be shown as below.



3.3 I/O Address Configuration and Linking to Module I/O's in TwinCAT 3

3.3.1 PLC Variable Creation in TwinCAT 3

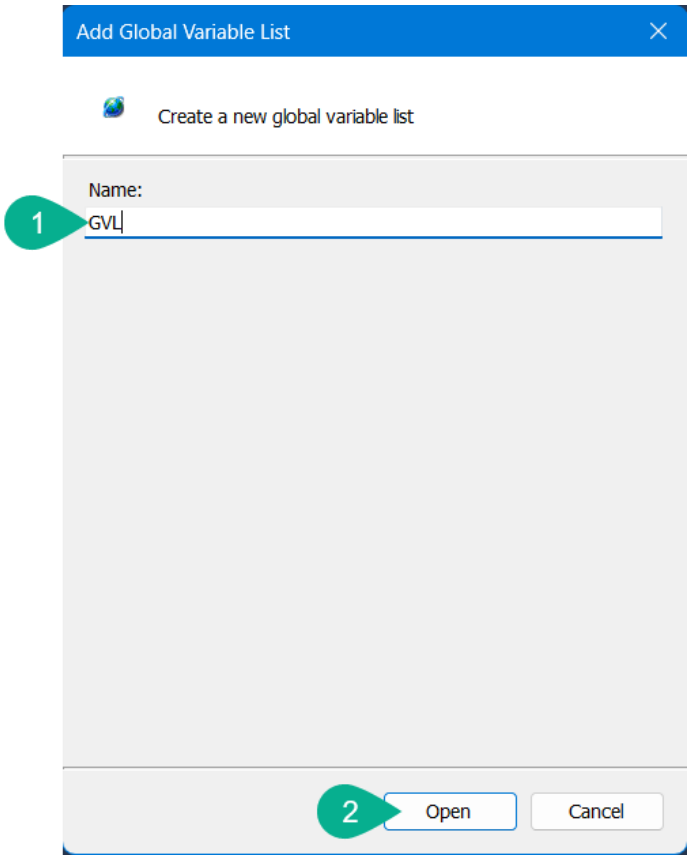
Step 1



1. Expand the **PLC** from Solution Explorer.
2. Expand the **VTEM_TESET** from PLC tree.
3. Expand the **VTEM_TESET Project** to view the PLC objects.
4. Right click on **GVLs** to view the Global variable options.
5. Click on **Add** from global variable options.
6. Click on **Global Variable List** from the Add option.

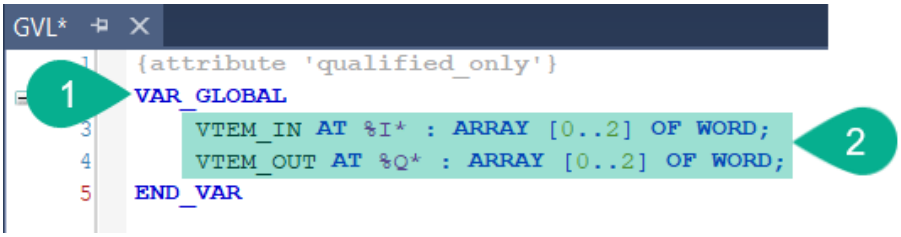
The Add Global Variable List tab will be displayed as shown below Step 2.

Step 2



1. Enter the **Global Variable List** Name on Name field.
2. Click on **Open** to view the created Global Variable list.

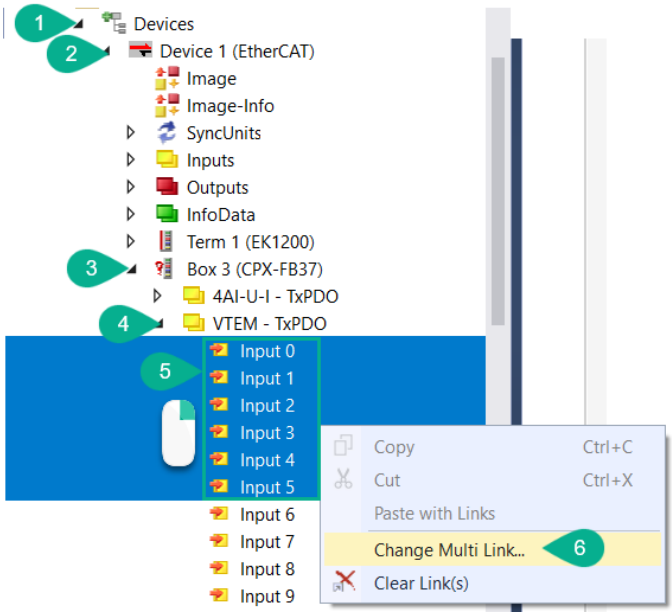
Step 3



1. Create the required variables under the **VAR_GLOBAL**.
2. The created global variables are shown above image.

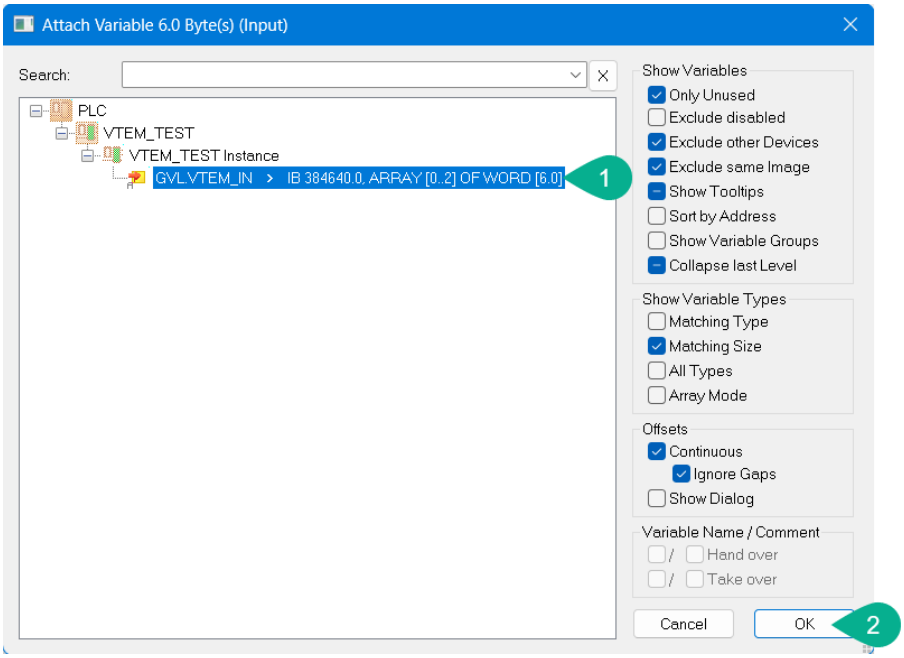
3.3.2 PLC Variables Link to Module I/Os

Step 1



1. Expand the **Devices** from Solution Explorer.
2. Expand the **Device 1 (EtherCAT)** from Device list.
3. Expand the **Box3 (CPX-FB37)** to view the Modules.
4. Expand the **VTEM-TxPDO** to view the Inputs.
5. Hold **Shift** and Select **Input 0** to **Input 5** and right click on the selected inputs.
6. Click on **Change Multi Link.**

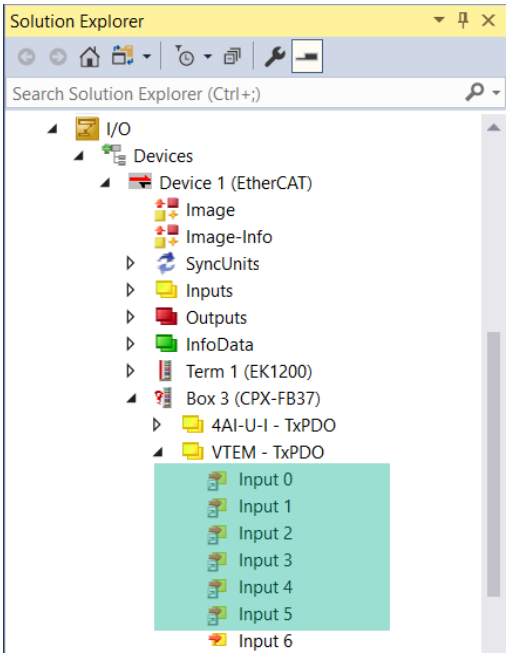
Step 2



1. Select the Input variable from PLC variable list.
2. Click on **OK** button to Link the PLC variable to Module input.

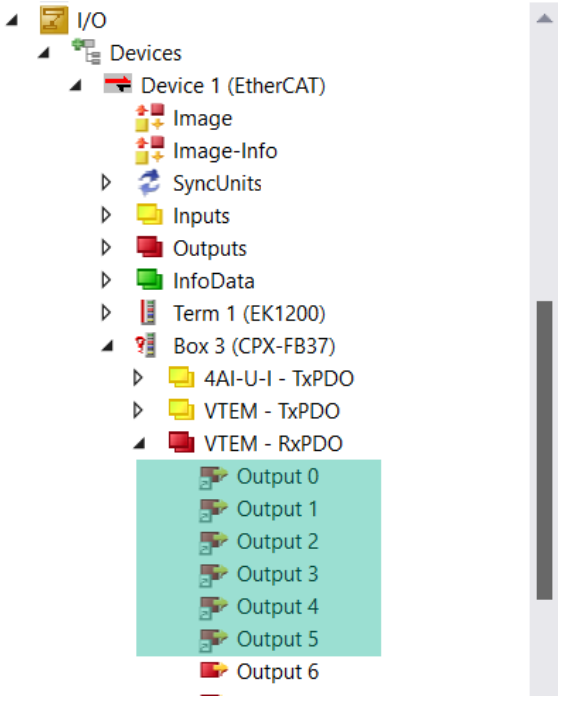
The linked Input variable will be displayed as shown below Step 3.

Step 3



Step 4

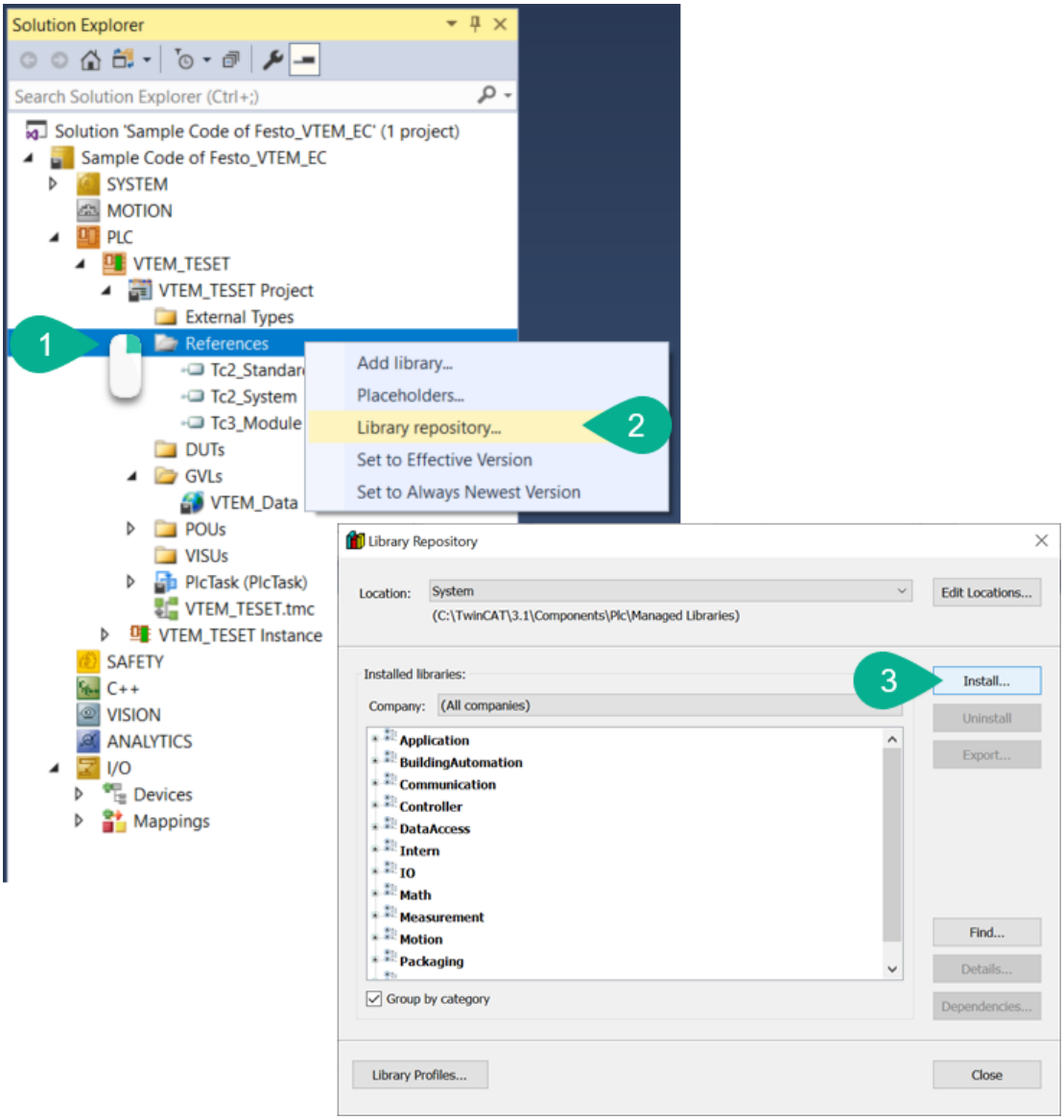
1. Follow the above Step 1 and Step 2 to link the PLC variable to Module output.
2. The linked Output variable will be displayed below.



3.4 Adding Festo_VTEM_EC Library to TwinCAT 3

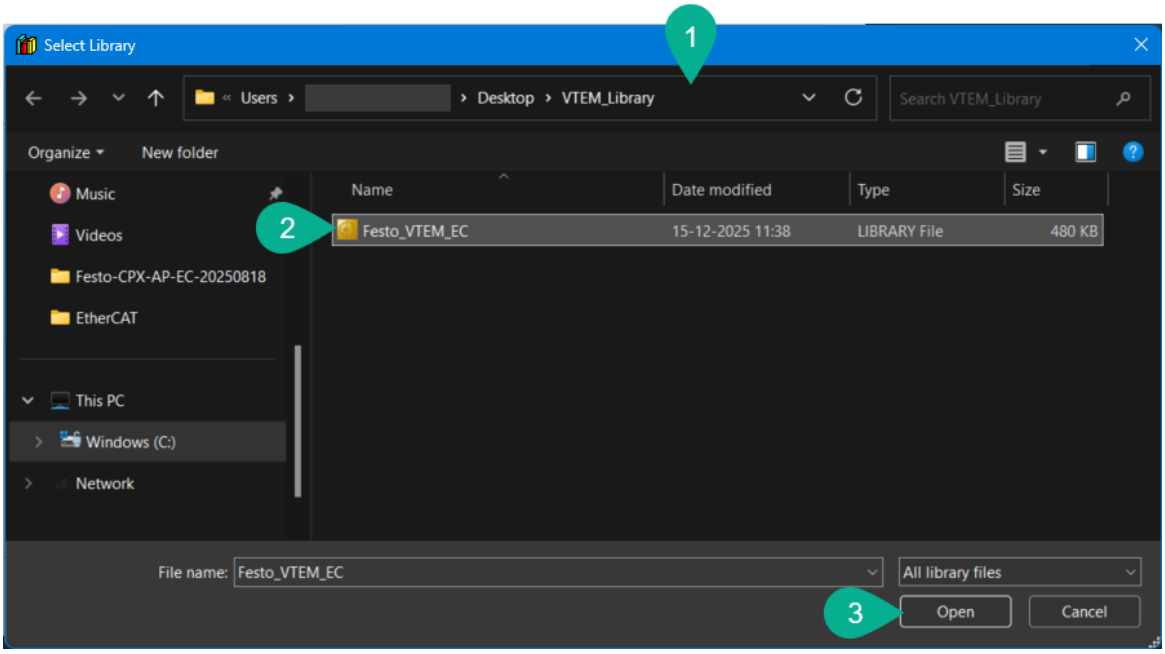
3.4.1 Importing Festo_VTEM_EC Library into TwinCAT 3

Step 1



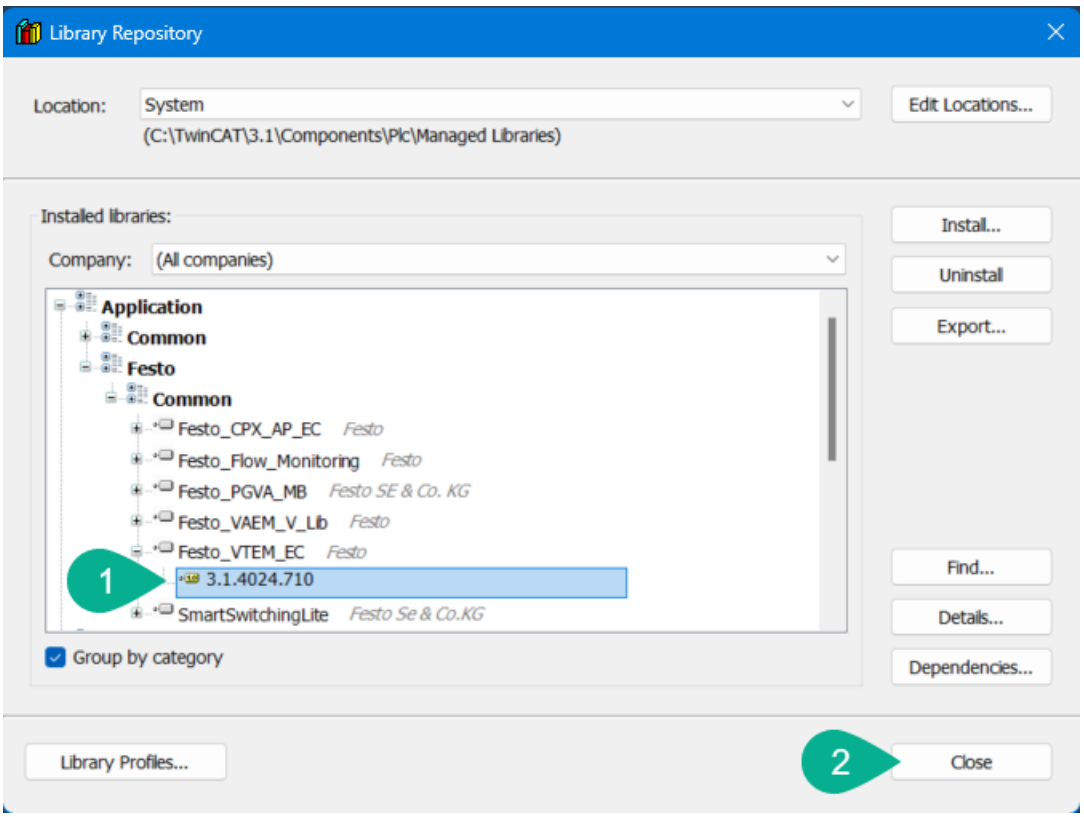
1. Right click on **References** under PLC objects.
2. Click on **Library repository** to open the Library repository window.
3. Click on **Install** button.

Step 2



1. Choose the **path** where the Library is saved in your system.
2. Select the **Library file** from Select library window.
3. Click on **Open** to install the library to TwinCAT 3 library repository .

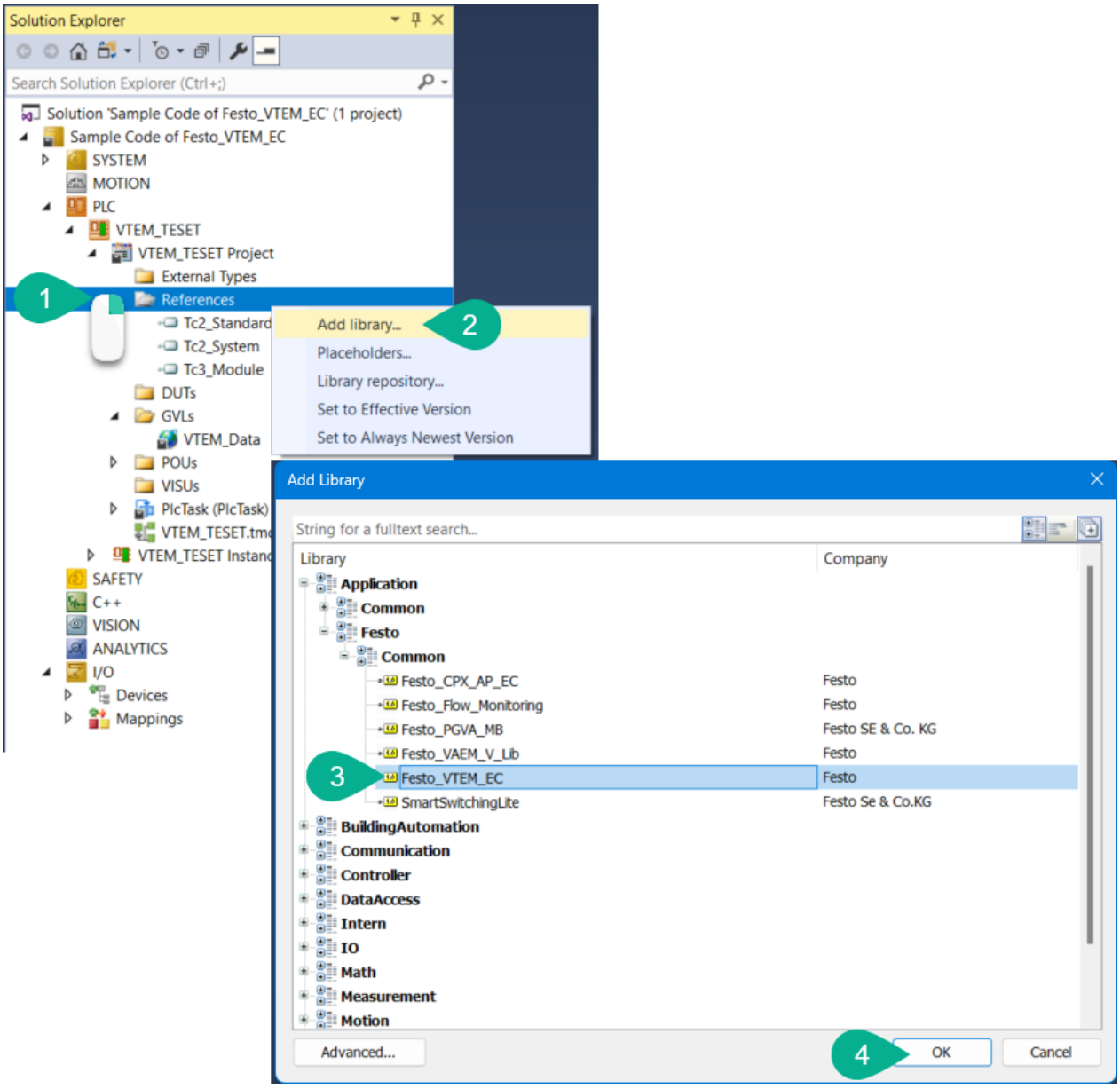
Step 3



1. The installed library comes under the **Application >> Festo >> Common >> Festo_VTEM_EC.**
2. Click on **Close** button to close the Library Repository window.

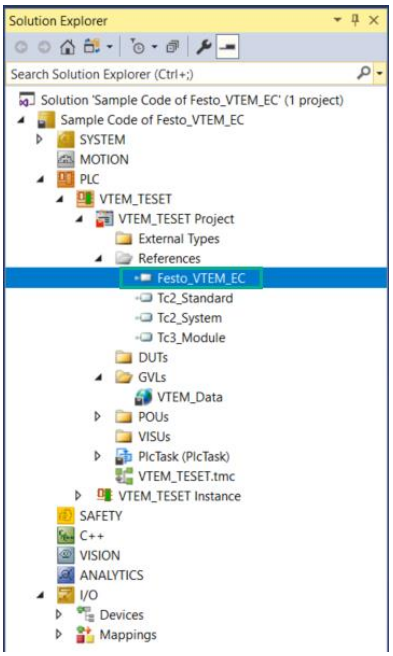
3.4.2 Adding Festo_VTEM_EC Library into TwinCAT Project

Step 1



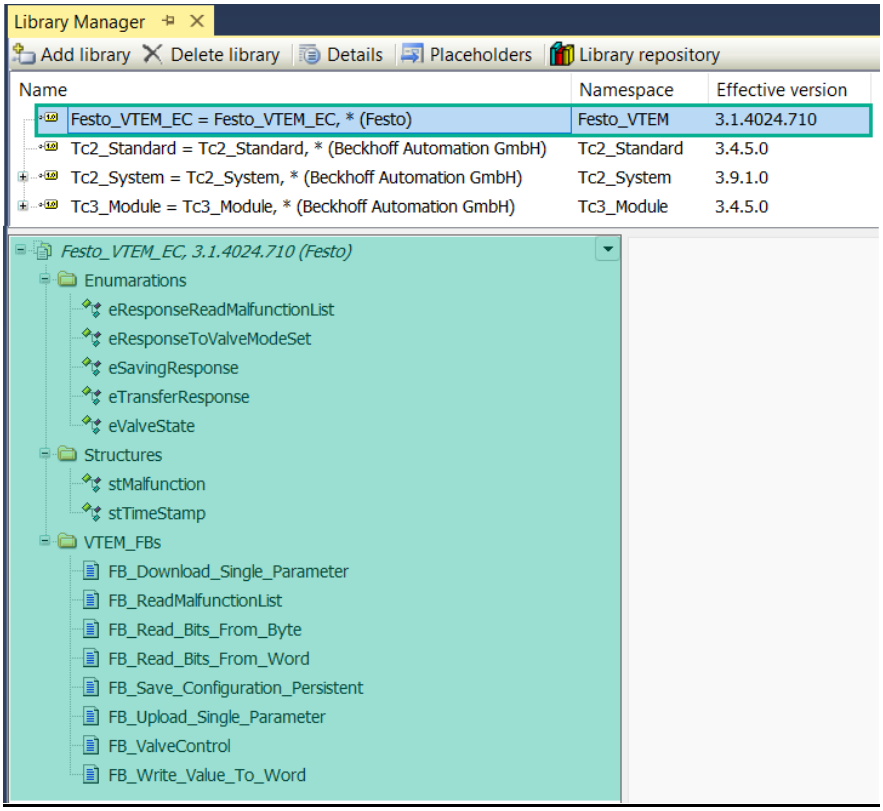
1. In the **Solution Explorer** pane, right click on **References** option under **PLC** menu.
2. Click on **Add Library...** option.
3. In **Add Library** window, browse for **Festo_VTEM_EC** library from installed library list (**Application >> Festo >> Common >> Festo_VTEM_EC**).
4. Click on **OK** button to add the library.

Step 2



In the **Solution Explorer** pane, the library will be added on **References** option as shown, double click on the added library to view its contents.

Step 3



Once the library is added successfully, the library elements will be available as shown above, highlighted in green color.

Function Blocks

- **FB_ValveControl** block is used to Control the Single valve in the motion terminal.
- **FB_Upload_Single_Parameter** block is used to read the motion terminal valve parameter.
- **FB_Download_Single_Parameter** block is used to write the motion terminal valve parameter.
- **FB_Save_Configuration** block is used to save the motion terminal valve parameter.
- **FB_ReadMalfunctionList** block is used to read the malfunction list of the VTEM motion terminal.

Structures

- stMalfunction
- stTimeStamp

Enumeration

- eResponseReadMalfunctionList
- eResponseToValveModeSet
- eSavingResponse
- eTransferResponse
- eValveState

Supporting Blocks

- **FB_Read_Bits_From_Byte** block is used to read the bits data from the byte.
- **FB_Read_Bits_From_Word** block is used to read the bits data from the word.
- **FB_Write_Value_To_Word** block is used to write the value to particular bits of the word.

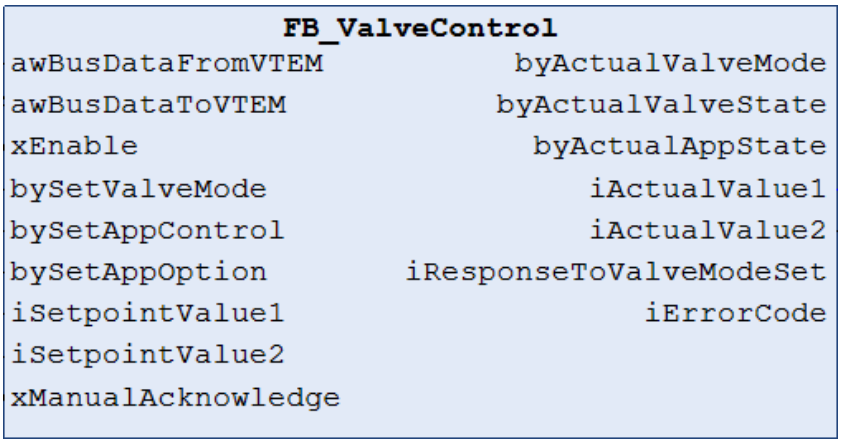
➔ **Note**

- **FB_Read_Bits_From_Byte**, **FB_Read_Bits_From_Word** and **FB_Write_Value_To_Word** are auxiliary function blocks.

4 Festo_VTEM_EC Library Function Blocks Description

4.1 FB_ValveControl

Function block **FB_ValveControl** serves to operate and control the Motion Apps. For this, the function block interprets the values at the inputs and generates the in-out data that is sent to the Motion Terminal. In parallel, the function block interprets the input data of a valve received from the Motion Terminal, classifies the information in it and issues it to the appropriate outputs. This means that the status of the function block outputs is updated at least one cycle after the change of the input values.



4.1.1 FB_ValveControl Block Input Variable

Input Variables	Data Type	Description
xEnable	BOOL	Input must be TRUE to be able to use the function block. At a change to FALSE, the function block stops any operation running on the valve (Valve Mode = valve inactive). The output iResponseToValveModeSet shows the value 1051 = FB_not_enabled after the function block was stopped successfully.
bySetValveMode	BYTE	Input for setting the valve mode. Permissible values are: 1 … 59 (ID Motion App) 60 (Teach-in run) 61 (End Motion App) 62 (Acknowledge inactive error), alternatively use input “xManualAcknowledge”
bySetAppControl	BYTE	Input for setting the Motion App control. The meaning is specific to the Motion App.
bySetAppOption	BYTE	Input for setting the Motion App setting. The meaning is specific to the Motion App. For Motion Apps #01 and #02, the enumeration eValveType_MA_01_02 can be used. For Motion App #02 there is a function block for composing the App Option (FB_AppOptionGenerator_MA_02)
iSetPointValue1	INT	Input for setpoint value 1: The meaning is specific to the Motion App.
iSetPointValue2	INT	Input for setpoint value 2: The meaning is specific to the Motion App.
xManualAcknowledge	BOOL	Inactive errors in valve status “not ready” can be acknowledged with a rising edge at this input (corresponds to specification “bySetValveMode := 62” --> Valve status becomes “configurable”).

Table 4.1: FB_ValveControl Block Input Variable descriptions

4.1.2 FB_ValveControl Block Output Variable

Output Variables	Data Type	Description
byActualValveMode	BYTE	Returned mode of the valve
byActualValveState	BYTE	Returned status of the valve
byActualAppState	BYTE	Returned status of the Motion App
iActualValue1	INT	Returned actual value 1
iActualValue2	INT	Returned actual value 2
iResponseToValveMode-Set	eResponseToValveModeSet (ENUM)	Response to the sent ValveMode (Input bySetValve-Mode). Refer Chapter 4.7.2
iErrorCode	INT	In case of an error (valve state = “failure” or “not ready”), the error code of the last error is at this output.

Table 4.2: FB_ValveControl Block Output Variable descriptions

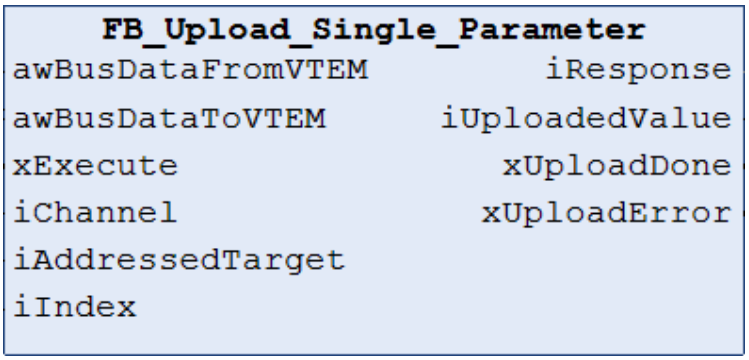
4.1.3 FB_ValveControl Block In-Out Variables

In-Out Variables	Data Type	Description
awBusDataFromVTEM	ARRAY [0..2] OF WORD	The 6 Bytes of input data received from the Motion Terminal must be applied to this input.
awBusDataToVTEM	ARRAY [0..2] OF WORD	The 6 Bytes of output data that must be sent to the Motion Terminal are available at this output.

Table 4.3: FB_ValveControl Block In-Out Variables descriptions

4.2 FB_Upload_Single_Parameter

The function block transmits a value via the transfer mode from the Motion Terminal to the PLC.



4.2.1 FB_Upload_Single_Parameter Block Input Variable

Input Variables	Data Type	Description
xExecute	BOOL	Rising edge at the input starts the transmission. If the “TRUE” signal is removed before transmission is completed (xUploadDone = TRUE), the transfer mode is terminated.
iChannel	INT	Transfer mode channel
iAddressedTarget	INT	Channel-specific address of the target from which the value to be transmitted is to be read.
iIndex	INT	Index for which the stored value is to be read out.

Table 4.4: FB_Upload_Single_Parameter Block Input Variable descriptions

4.2.2 FB_Upload_Single_Parameter Block Output Variable

Output Variables	Data Type	Description
iResponse	eTransferResponse (ENUM)	Response to the status of the transmission. Refer Chapter 4.7.4
iUploadedValue	UINT	Read out value
xUploadDone	BOOL	Output becomes TRUE when the transfer could be concluded without an error
xUploadError	BOOL	Output becomes TRUE when errors occurred during the transfer.

Table 4.5: FB_Upload_Single_Parameter Block Output Variable descriptions

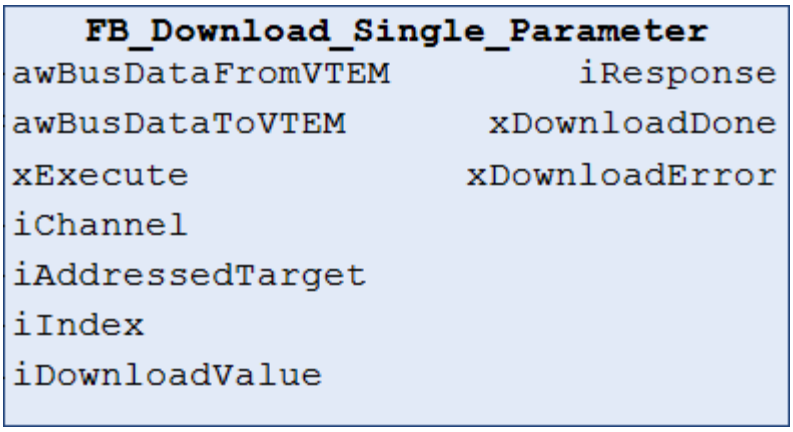
4.2.3 FB_Upload_Single_Parameter Block In-Out Variables

In-Out Variables	Data Type	Description
awBusDataFromVTEM	ARRAY [0..2] OF WORD	The 6 Bytes of input data received from the Motion Terminal must be applied to this input.
awBusDataToVTEM	ARRAY [0..2] OF WORD	The 6 Bytes of output data that must be sent to the Motion Terminal are available at this output.

Table 4.6: FB_Upload_Single_Parameter Block In-Out Variables descriptions

4.3 FB_Download_Single_Parameter

The function block transmits a value via the transfer mode from the PLC to the Motion Terminal. It can only be used for writable values.



4.3.1 FB_Download_Single_Parameter Block Input Variable

Input Variables	Data Type	Description
xExecute	BOOL	Rising edge at the input starts the transmission. If the “TRUE” signal is removed before transmission is completed (xDownloadDone = TRUE), the transfer mode is terminated.
iChannel	INT	Transfer mode channel
iAddressedTarget	INT	Channel-specific address of the target into which the value to be transmitted is to be written.
iIndex	INT	Index of the parameter that is to be set on the transmitted value.
iDownloadValue	INT	Value on which the parameter is to be set.

Table 4.7: FB_Download_Single_Parameter Block Input Variable descriptions

4.3.2 FB_Download_Single_Parameter Block Output Variable

Output Variables	Data Type	Description
iResponse	eTransferResponse (ENUM)	Response to the status of the transmission. Refer Chapter 4.7.4
xDownloadDone	BOOL	Output becomes TRUE when the transfer could be concluded without an error.
xDownloadDone	BOOL	Output becomes TRUE when errors occurred during the transfer.

Table 4.8: FB_Download_Single_Parameter Block Output Variable descriptions

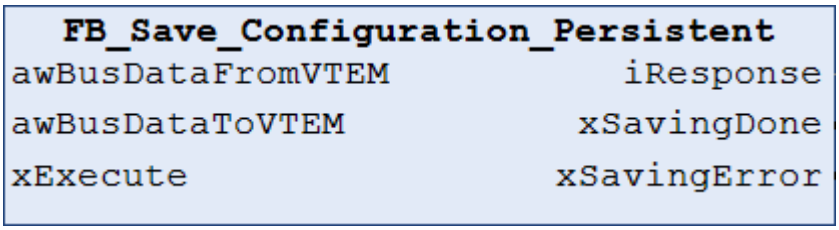
4.3.3 FB_Download_Single_Parameter Block In-Out Variables

In-Out Variables	Data Type	Description
awBusDataFromVTEM	ARRAY [0..2] OF WORD	The 6 Bytes of input data received from the Motion Terminal must be applied to this input.
awBusDataToVTEM	ARRAY [0..2] OF WORD	The 6 Bytes of output data that must be sent to the Motion Terminal are available at this output.

Table 4.9: FB_Download_Single_Parameter Block In-Out Variables descriptions

4.4 FB_Save_Configuration

The function block serves for the simple call-up of function “Save configuration persistent”. The data for the valve currently being parameterized are saved permanently in the controller of the Motion Terminal.



4.4.1 FB_Save_Configuration Block Input Variable

Input Variables	Data Type	Description
xExecute	BOOL	Rising edge at the input starts a saving process. A new saving process therefore requires a new rising edge. If the “TRUE” signal is removed before the save operation is completed (xSavingDone = TRUE), the transfer mode is terminated.

Table 4.10: FB_Save_Configuration Block Input Variable descriptions

4.4.2 FB_Save_Configuration Block Output Variable

Output Variables	Data Type	Description
iResponse	eSavingResponse (ENUM)	Response to the status of saving process. Refer Chapter 4.7.3
xSavingDone	BOOL	The output becomes “TRUE” when the saving process is completed.
xSavingError	BOOL	The output becomes “TRUE” when an error occurred during the saving process.

Table 4.11: FB_Save_Configuration Block Output Variable descriptions

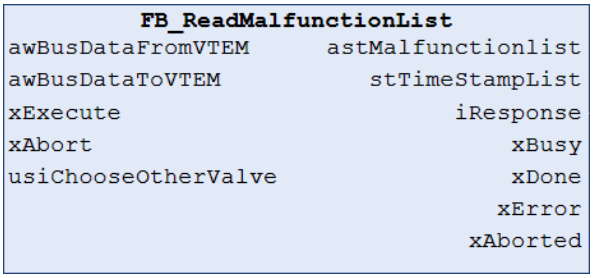
4.4.3 FB_Save_Configuration Block In-Out Variables

In-Out Variables	Data Type	Description
awBusDataFromVTEM	ARRAY [0..2] OF WORD	The 6 Bytes of input data received from the Motion Terminal must be applied to this input.
awBusDataToVTEM	ARRAY [0..2] OF WORD	The 6 Bytes of output data that must be sent to the Motion Terminal are available at this output.

Table 4.12: FB_Save_Configuration Block In-Out Variables descriptions

4.5 FB_ReadMalfunctionList

The function block **FB_ReadMalfunctionList** automatically reads the contents of the malfunction list of a valve and passes it to the output **stMalfunctionlist**. The input **uiChooseOtherValve** defines from which valve the malfunction list should be read. This way the process data of any other slot can be used for this purpose.



4.5.1 FB_ReadMalfunctionList Block Input Variable

Input Variables	Data Type	Description
xExecute	BOOL	TRUE signal at the input starts the transmission. If the TRUE signal is removed, the transfer mode is terminated and the function block is reset as soon as the transfer process has been completed successfully (iResponse = “done” or “no_entries_in_malfunction_list”) or with an error (xError = TRUE).
xAbort	BOOL	TRUE signal at the input terminates the process in the function block (iResponse = “aborting”). The transfer mode is terminated if necessary and the valve is set to the “inactive” state (valve mode = 61).
usiChooseOtherValve	USINT	Select the valve whose malfunction list is to be read. 0: Read the malfunction list of the valve whose process data the function block uses. 100 … 107: Read the malfunction list of the valve in slot 0 (100) … 7 (107).

Table 4.13: FB_ReadMalfunctionList Block Input Variable descriptions

4.5.2 FB_ReadMalfunctionList Block Output Variable

Output Variables	Data Type	Description
astMalfunctionlist	ARRY [1..40] OF stMalfunction	Content of the malfunction list The content of the array is complete as soon as xDone = TRUE. The output is reset as soon as xAbort = FALSE and xExecute = FALSE. Refer Chapter 4.6.1
stTimeStampList	stTimeStamp (Struct)	Creation time of the malfunction list. Refer Chapter 4.6.2
iResponse	eResponseReadMalfunctionList (ENUM)	Response to the status of the transmission. Refer Chapter 4.7.1
xBusy	BOOL	Output is TRUE as long as the function block executes an action.
xDone	BOOL	Output is TRUE when the transfer could be concluded without an error (even if the malfunction list is empty).
xError	BOOL	Output is TRUE if an error occurred during the transmission process
xAborted	BOOL	Output is TRUE if the transmission was aborted by the input “xAbort”. Output is reset if xExecute and xAbort are FALSE at the same time.

Table 4.14: FB_ReadMalfunctionList Block Output Variable descriptions

4.5.3 FB_ReadMalfunctionList Block In-Out Variable

In-Out Variables	Data Type	Description
awBusDataFromVTEM	ARRAY [0..2] OF WORD	The 6 Bytes of input data received from the Motion Terminal must be applied to this input.
awBusDataToVTEM	ARRAY [0..2] OF WORD	The 6 Bytes of output data that must be sent to the Motion Terminal are available at this output.

Table 4.15: FB_ReadMalfunctionList Block In-Out Variable descriptions

4.6 Structure Data Types

4.6.1 stMalfunction

Structure Variables	Data Type	Description
usiMalfunctionCode	USINT	Code of malfunction list entry
usiMalfunctionSubcode	USINT	Subcode of malfunction list entry
usiMalfunctionClassification	USINT	Classification of malfunction list entry: 1: active error 2: inactive error 3: warning
uiHours	UINT	Time stamp (hours) of the malfunction list entry
usiMinutes	USINT	Time stamp (minutes) of the malfunction list entry
usiSeconds	USINT	Time stamp (seconds) of the malfunction list entry
uiMilliseconds	UINT	Time stamp (milliseconds) of the malfunction list entry

Table 4.16: stMalfunction descriptions

4.6.2 stTimeStamp

Structure Variables	Data Type	Description
uiHours	UINT	Time stamp (hours) of the malfunction list entry
usiMinutes	USINT	Time stamp (minutes) of the malfunction list entry
usiSeconds	USINT	Time stamp (seconds) of the malfunction list entry
uiMilliseconds	UINT	Time stamp (milliseconds) of the malfunction list entry

Table 4.17: stTimeStamp descriptions

4.7 Enumeration Data Types

4.7.1 eResponseReadMalfunctionList

Enumeration Value	Enumeration Text
0	idle
1	setting_valve_to_inactive
2	getting_malfunction_count
3	transferring
4	malfunction_list_read_complete
5	no_entries_in_malfunction_list

Enumeration Value	Enumeration Text
6	error_while_transmitting_data_to_vtem
7	aborting
8	resetting
10	FB_needs_rising_edge_xExecute
100	reserved
101	invalid_SetValveMode
102	invalid_SetAppControl
103	invalid_SetAppOption
104	invalid_SetPointValue1
105	invalid_SetPointValue2
106	invalid_Configuration
110	access_denied_WebConfig
111	access_denied_saving_parameterization
112	no_license_for_this_MA
113	all_licenses_in_use
114	invalid_license_file
115	no_valve_detected
116	valve_self_calibration_running
201	invalid_Channel
202	invalid_TransferControl
203	invalid_usiChooseOtherValve
204	invalid_Index
205	invalid_TransferValue
206	no_valve_detected_at_selected_slot
207	access_denied_DataInUse
208	no_data_empty_slot

Table 4.18: eResponseReadMalfunctionList enumeration values

4.7.2 eResponseToValveModeSet

Enumeration Value	Enumeration Text
0	reserved
1	invalid_SetValveMode
2	invalid_SetAppOption
3	invalid_SetAppControl
4	invalid_SetPointValue1
5	invalid_SetPointValue2
6	invalid_configuration
10	access_denied_WebConfig
11	access_denied_saving_parameterization
12	no_license_for_this_MA
13	all_licenses_in_use
14	invalid_license_file
15	no_valve_detected
16	valve_self_calibration_running
1020	no_TransferMode_with_this_FB
1031	preparing_for_start
1032	starting
1033	MA_running
1034	TransferMode_running
1035	stopping
1036	all_MAs_stopped
1037	acknowledge_needed
1038	failure
1040	preparing_for_acknowledge
1041	acknowledging
1042	all_errors_acknowledged
1043	failure_still_active
1050	no_communication
1051	FB_not_enabled

Table 4.19: eResponseToValveModeSet enumeration values

4.7.3 eSavingResponse

Enumeration Value	Enumeration Text
0	reserved
1	saving_in_progress
2	saving_successful
3	saving_not_possible
4	saving_failed
10	access_denied_WebConfig
11	access_denied_saving
15	no_valve_detected
20	preparing_to_save
21	ready_to_save
25	FB_needs_rising_edge_xExecute

Table 4.20: eSavingResponse enumeration values

4.7.4 eTransferResponse

Enumeration Value	Enumeration Text
0	reserved
1	invalid_Channel
2	invalid_TransferControl
3	invalid_AddressedTarget
4	invalid_Index
5	invalid_TransferValue
6	invalid_Combination
7	access_denied_DataInUse
8	no_data_empty_slot
9	invalid_ParameterSet
10	access_denied_WebConfig
11	access_denied_saving
15	no_valve_detected
20	reading_transfer mode
21	ready_for_new_transfer
22	preparing_transfer

Enumeration Value	Enumeration Text
23	transferring
24	transfer_successful
25	FB_needs_rising_edge_xExecute
26	Valve_Mode_is_not_61_inactive
27	invalid_firmware_version
28	communication_failed
29	invalid_module_number
30	resetting
31	aborting

Table 4.21: eTransferResponse enumeration values

4.7.5 eValveState

Enumeration Value	Enumeration Text
0	NotReady
1	Configurable
2	Running
3	Failure

Table 4.22: eValveState enumeration values

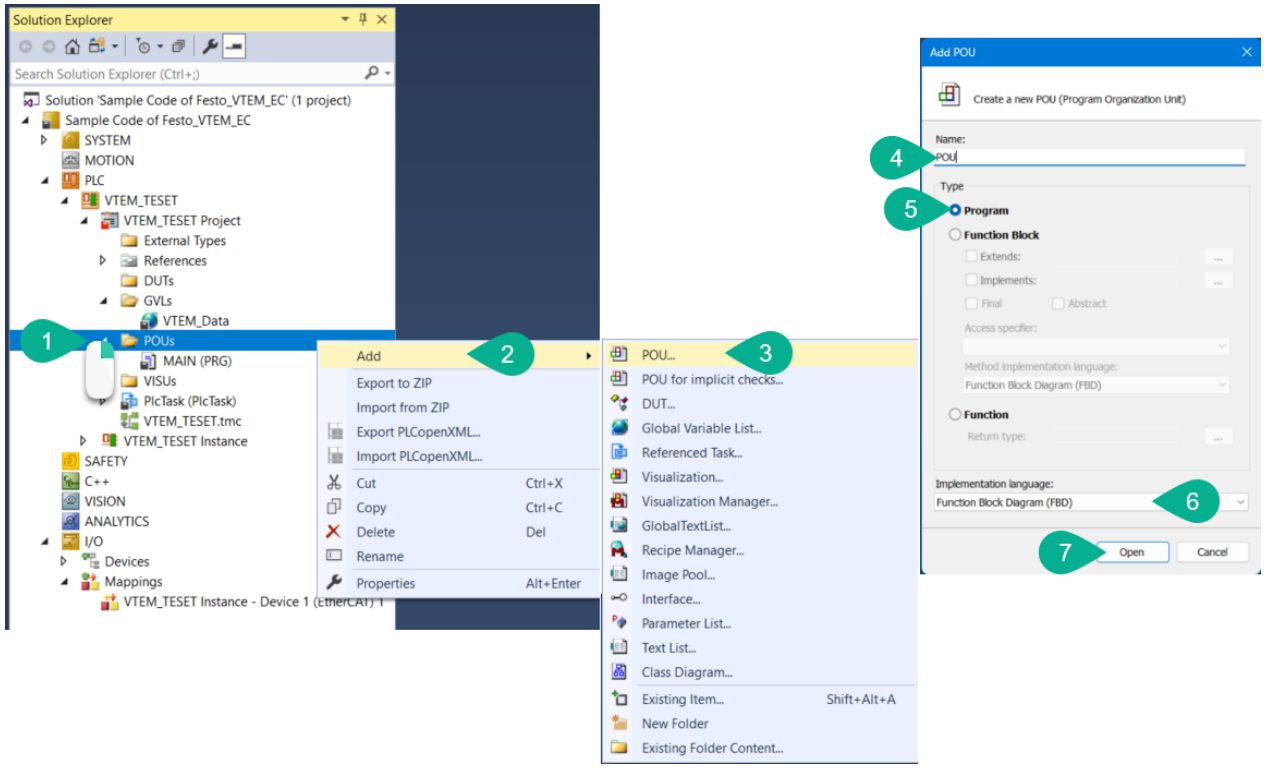
5 Configuration of VTEM Function Blocks in TwinCAT 3

5.1 Adding POU in Function Block Diagram Language

TwinCAT will add **MAIN (PRG)** in **ST (Structured Text) Language** by default when creating a new project.

This chapter explains how to create an **FBD (Function Block Diagram)** program in the TwinCAT 3 programming environment and execute it.

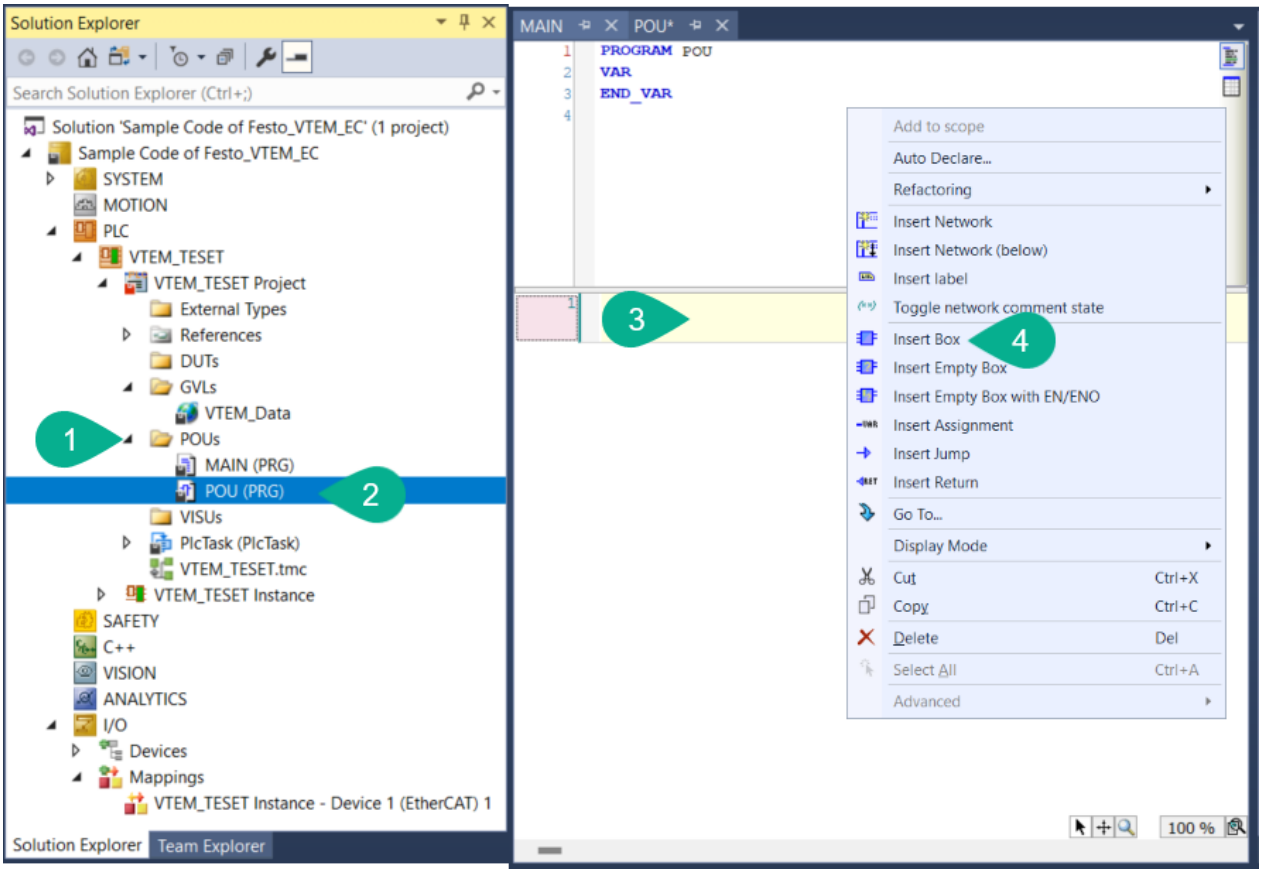
Step 1



1. In the **Solution Explorer** pane, right-click on the **POUs** folder.
2. Select the **Add** option from the available list.
3. Click on the **POU...** option from the available list to add the new Program Organization Unit (POU) to the project.
4. In the **Add POU** window, enter the required name for the POU in the **Name** section.
5. Select the **Type** of POU as **Program**, as shown in the above image.
6. Select **Function Block Diagram (FBD)** as the **Implementation Language** from the drop-down list.
7. Click the **Open** button to create the POU.

5.2 Configuring the Festo_VTEM_EC Function Block

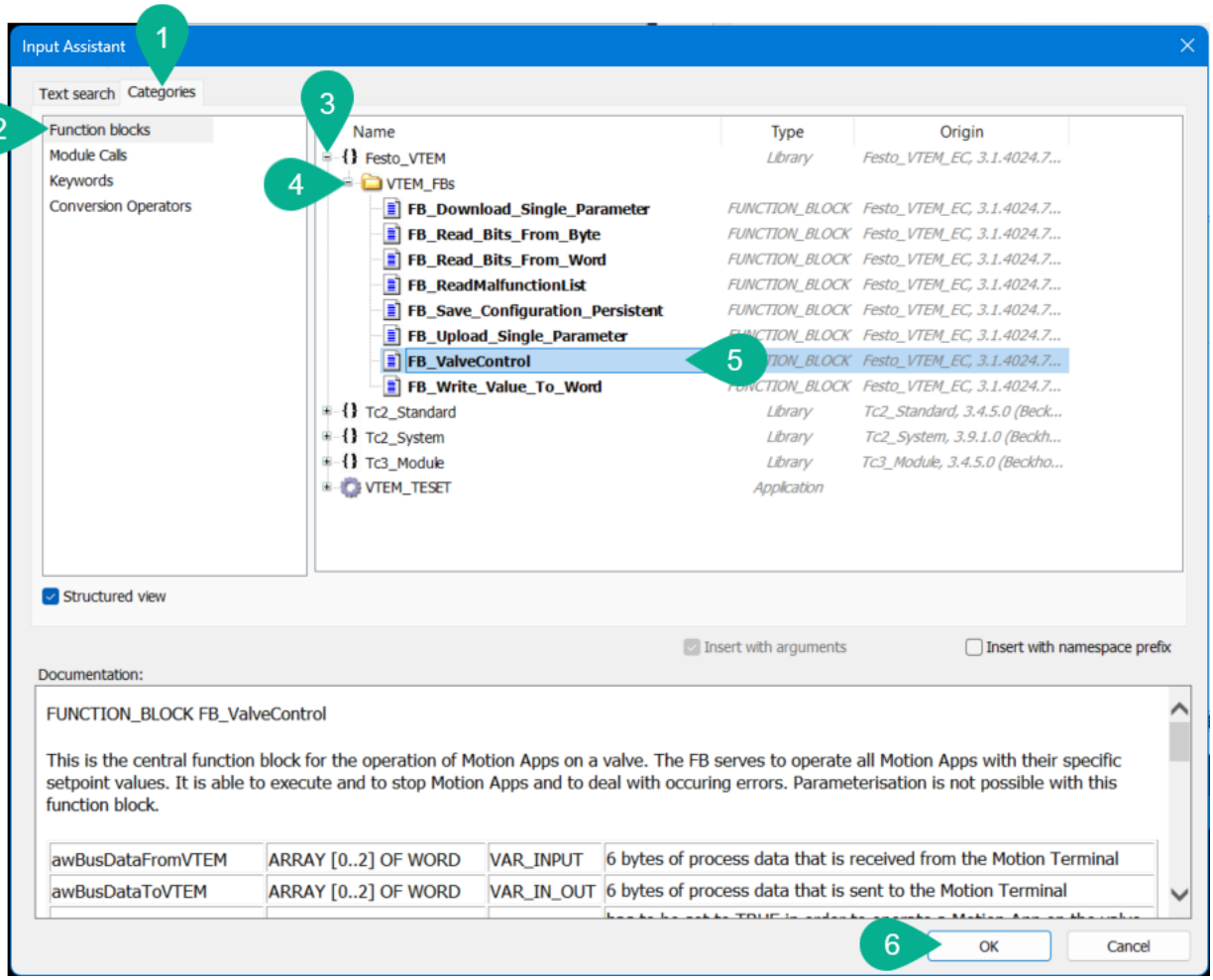
Step 1



1. Expand the **POUs** from PLC objects.
2. Double click on **Main** from POUs list.
3. Right click on the **Network** from Main window.
4. Click on **Box** to open the Input Assistant window.

The opened Input Assistant window will be displayed as show below in Step 2.

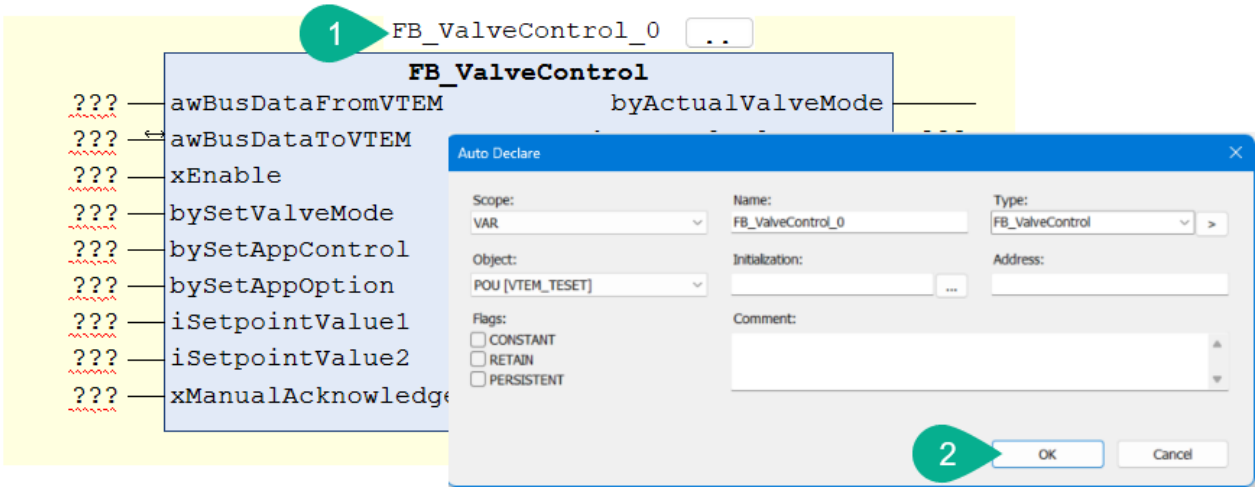
Step 2



1. Click on **Categories** tab in the Input Assistant window.
2. Click on **Function blocks** from Categories tab.
3. Expand **Festo_VTEM** .
4. Expand **POUs** to view the Function blocks.
5. Select **FB_ValveControl**.
6. Click on **OK** button to call the function block.

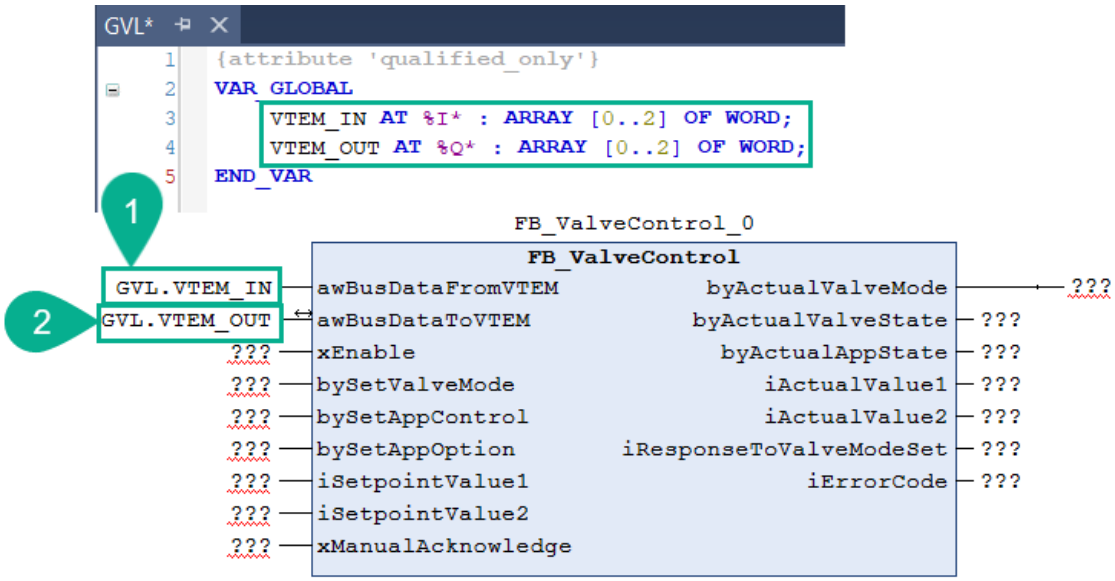
The called function block is shown below in Step 3.

Step 3



1. Enter the name of the FB instance & press enter key for Auto declare the tag name.
2. Click **OK** button from the auto declare window.

Step 4

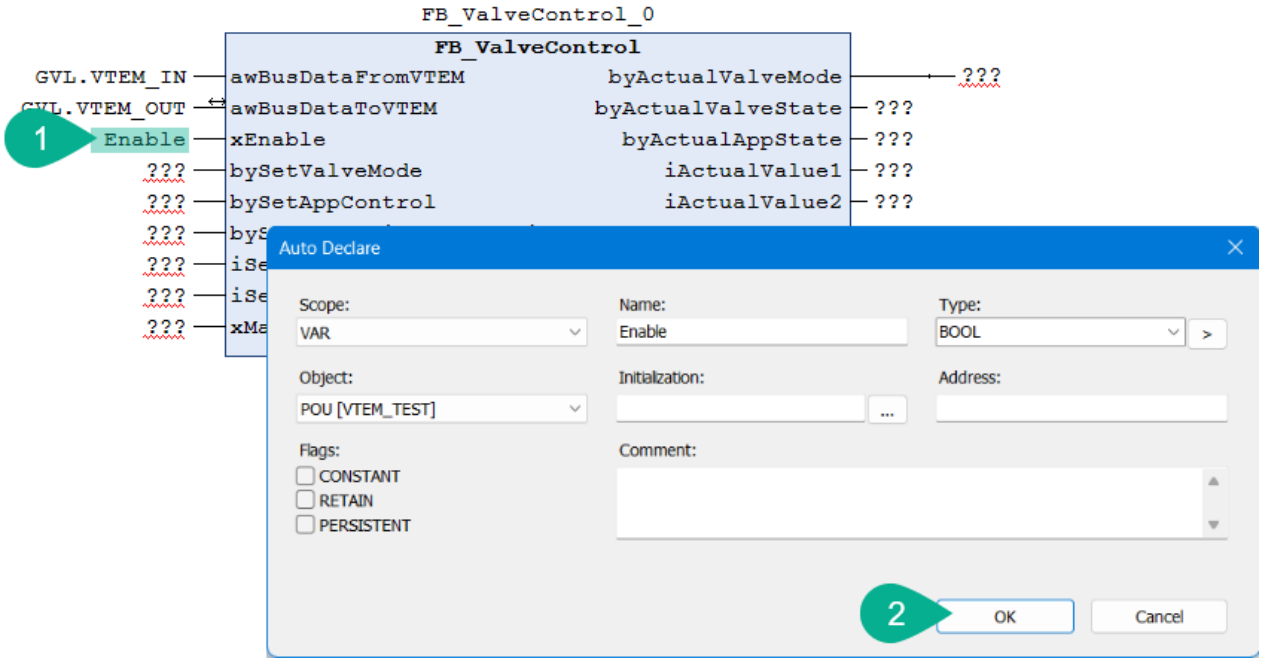


1. Enter **GVL.VTEM_IN** in awBusDataFromVTEM to assign the global variable input tag.
2. Enter **GVL.VTEM_OUT** in awBusDataToVTEM to assign the global variable output tag.

Note

- The user must take the input and output tags from the global variable list, which are linked to device I/Os.. Refer Chapter 3.3.1

Step 5

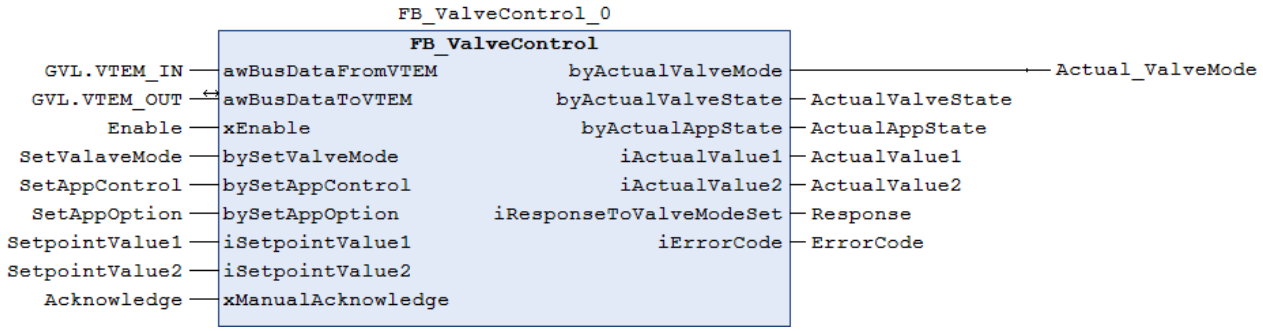


1. Enter the tag name for **xEnable** input tag & press enter to open Auto Declare window.
2. Click **OK** to confirm the entered tag name.

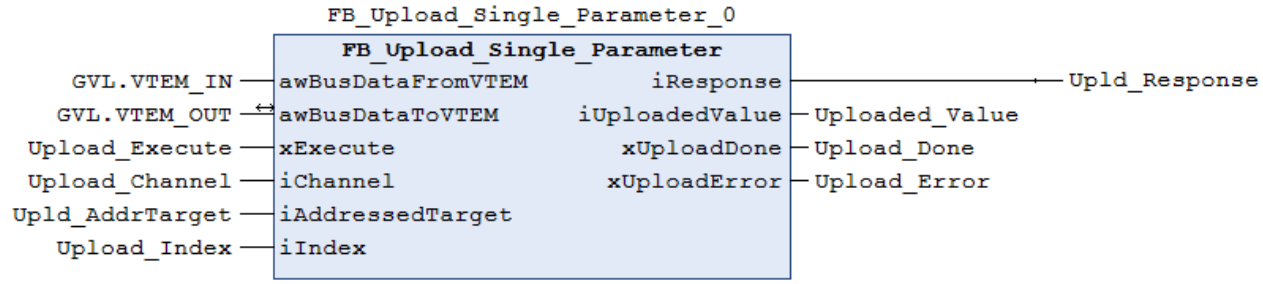
Note

- Repeat Step 5 for different input & output tags in function block.

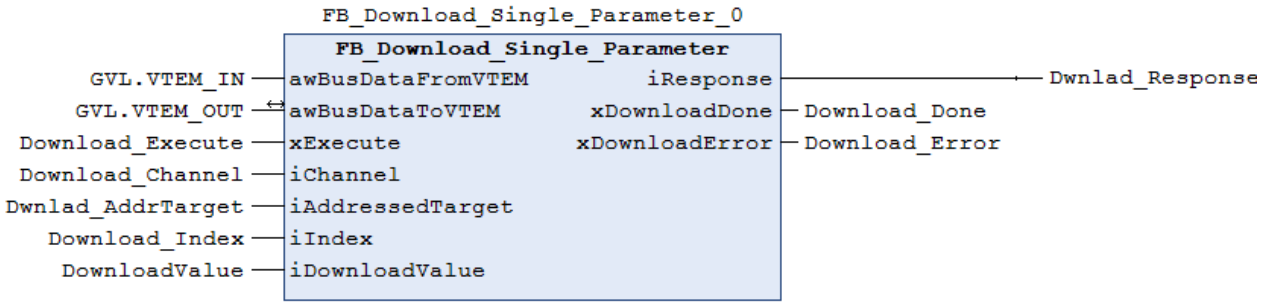
Once all the IO assignments are complete, the **FB_ValveControl** block will appear as shown below.



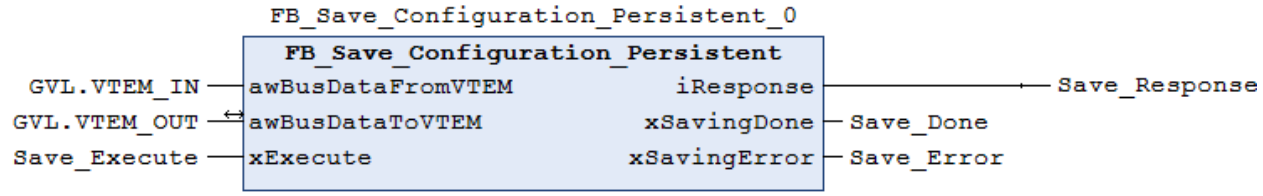
Once all the IO assignments are complete, the **FB_Upload_Single_Parameter** block will appear as shown below.



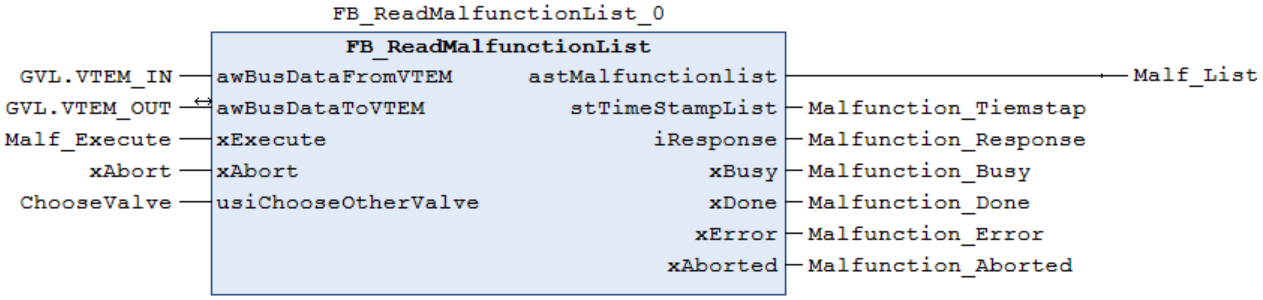
Once all the IO assignments are complete, the **FB_Download_Single_Parameter** block will appear as shown below.



Once all the IO assignments are complete, the **FB_Save_Configuration** block will appear as shown below.



Once all the IO assignments are complete, the **FB_ReadMalfunctionList** block will appear as shown below.



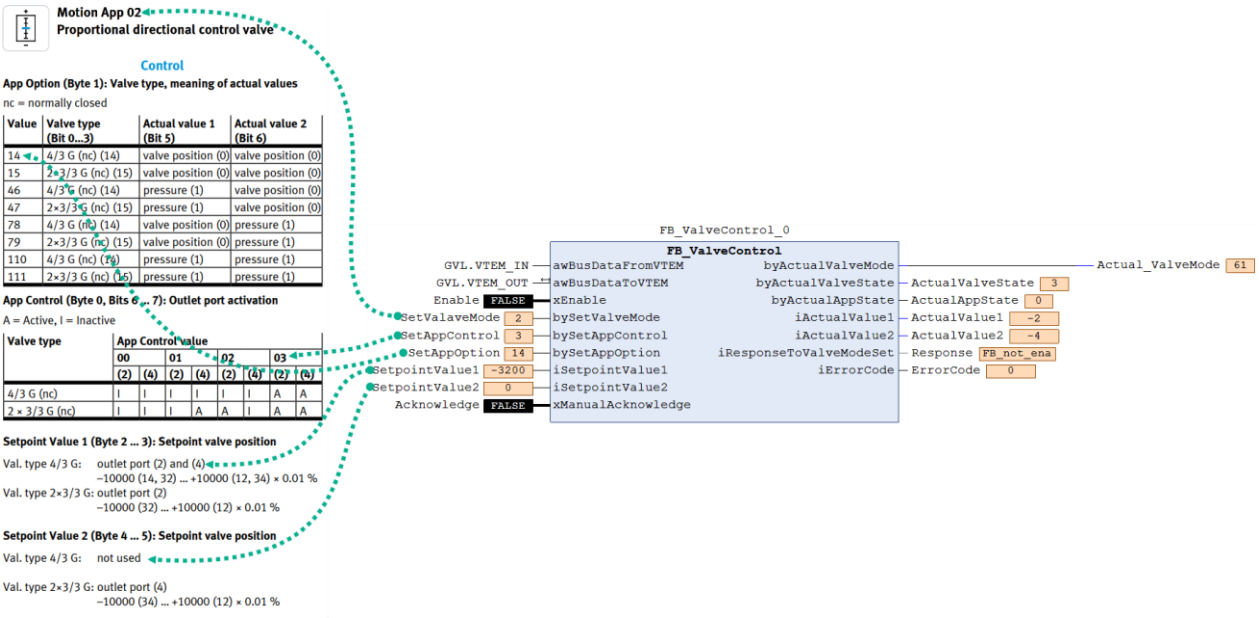
6 Sample Code for Festo_VTEM_EC Library Function Blocks

This sample code is used to control, read/write, save parameters and obtain diagnostic data from a VTEM Motion Terminal module connected to Beckhoff PLC through EtherCAT network using **Festo_VTEM_EC** library FBs.

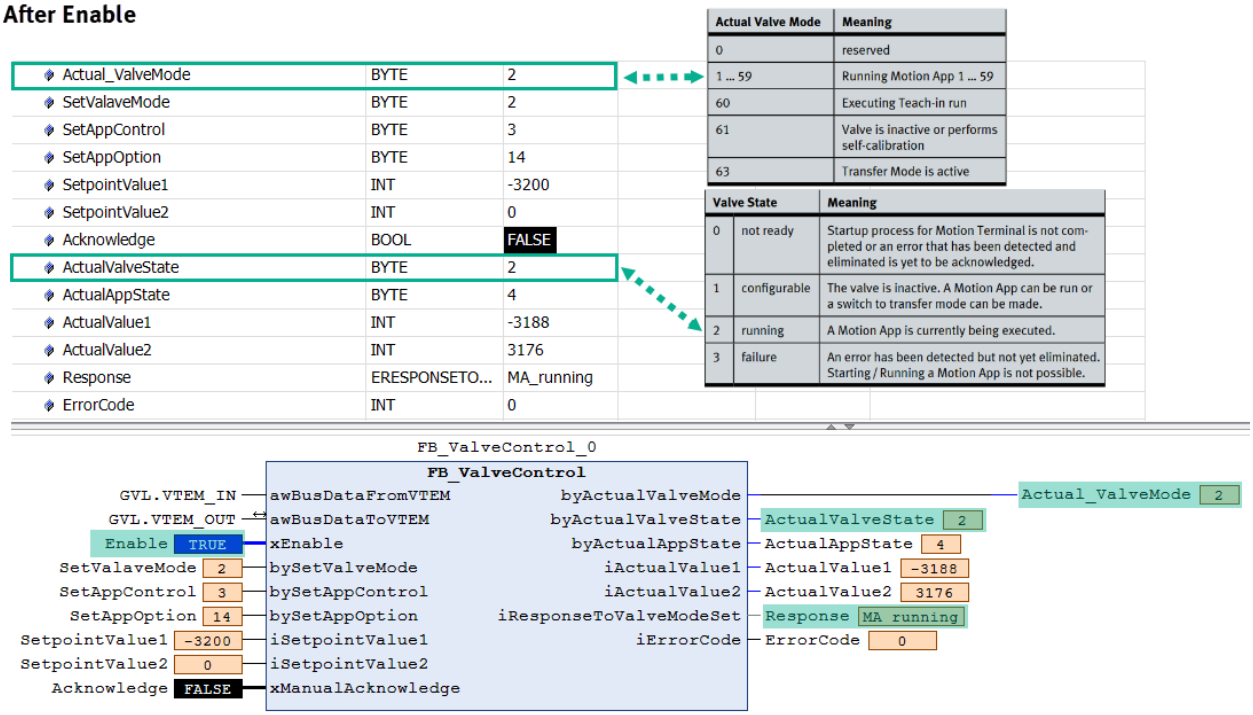
6.1 Sample Code for Controlling the VTEM Motion Terminal Valve Using FB_ValveControl

The sample code is used to control a specific valve using FB_ValveControl function block.

Before Enable



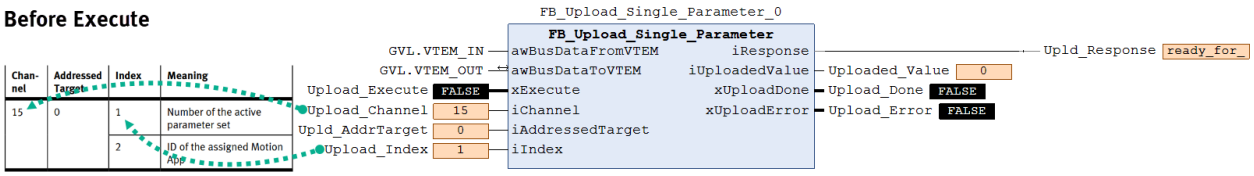
After Enable

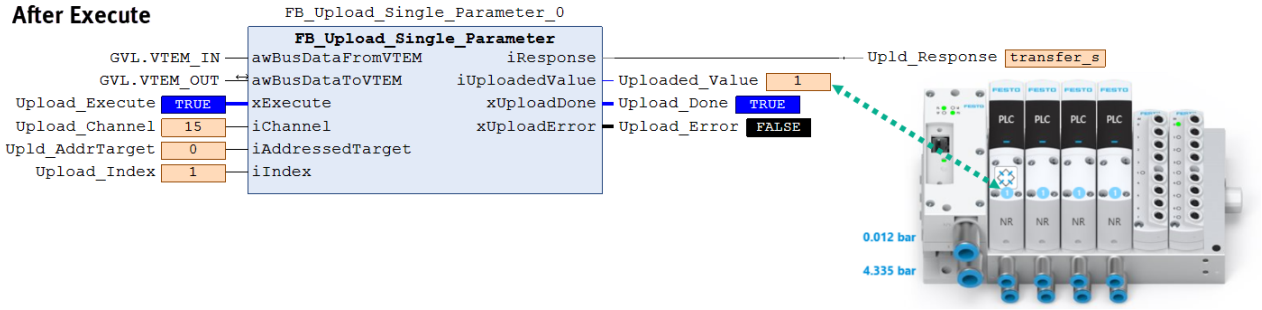


1. Enter the Motion App number in the **wSetValveMode** input tag.

2. Enter the App control value in the **wSetAppControl** input tag.
3. Enter the App option value in the **wSetAppOption** input tag.
4. Enter the setpoint value 1 in the **iSetpointValue1** input tag.
5. Enter the setpoint value 2 in the **iSetpointValue2** input tag.
6. Once all input tags are set, toggle the **xEnable** tag to start valve control.
7. If **iErrorCode** shows an error, execute the acknowledge function by setting the **xManualAcknowledge** control tag. Then enable the block again.
8. After the function block is enabled, the user will receive feedback in the appropriate tags: **byActualValveMode**, **byActualValveState**, **byActualAppState**, **iActualValue1**, **iActualValue2**.
9. The response to the valve-mode set command is available in the **iResponseToValveModeSet** monitor tag. (Refer Chapter 4.7.2)
10. If an error occurs, the error code is available in the **iErrorCode** monitor tag.

6.2 Sample Code for Reading the Parameter Set value of the Valve from the VTEM Motion Terminal Valve using FB_Upload_Single_Parameter





1. Enter the channel number as **15** in the **iChannel** input tag.
2. Enter **0** in the **iAddressedTarget** input tag.
3. Enter index number as **1** in the **iIndex** input tag.
4. Once all input tags are set, toggle the **xExecute** tag to start the upload process.
5. When the upload succeeds, the monitor tag **xUploadDone** becomes true.
6. The uploaded data is available in the **iUploadedValue** monitor tag . This value is the parameter set value, which will be **1** as shown in the image above.
7. The response of the upload process is available in the **iResponse** monitor tag. (Refer Chapter 4.7.4)
8. If an error occurs, the **xUploadError** monitor tag indicates it.

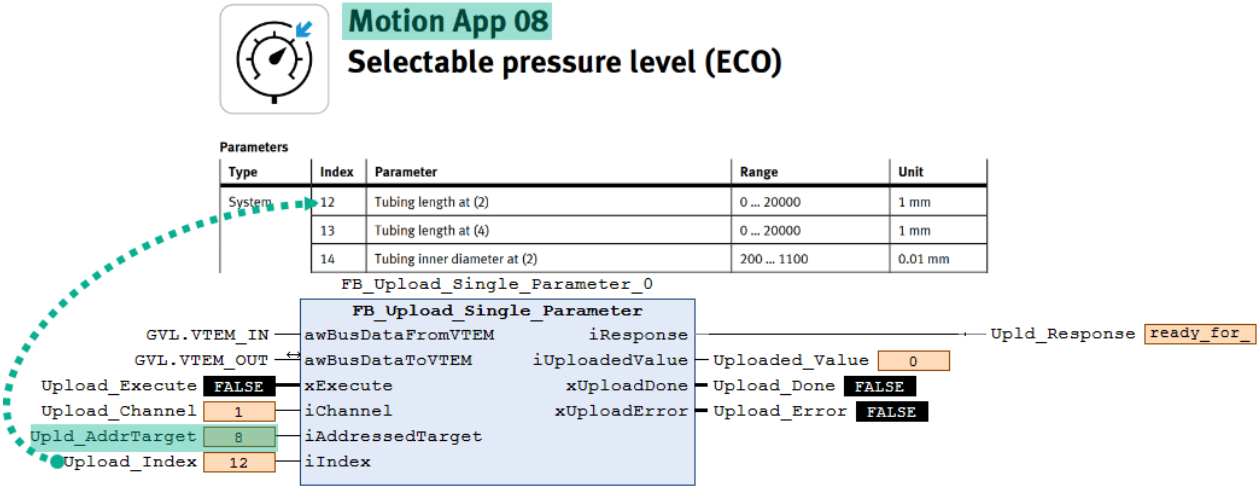
Note

- User can change the program set value for the valve using **FB_Download_Single_Parameter**, follow similar steps as shown in this chapter.

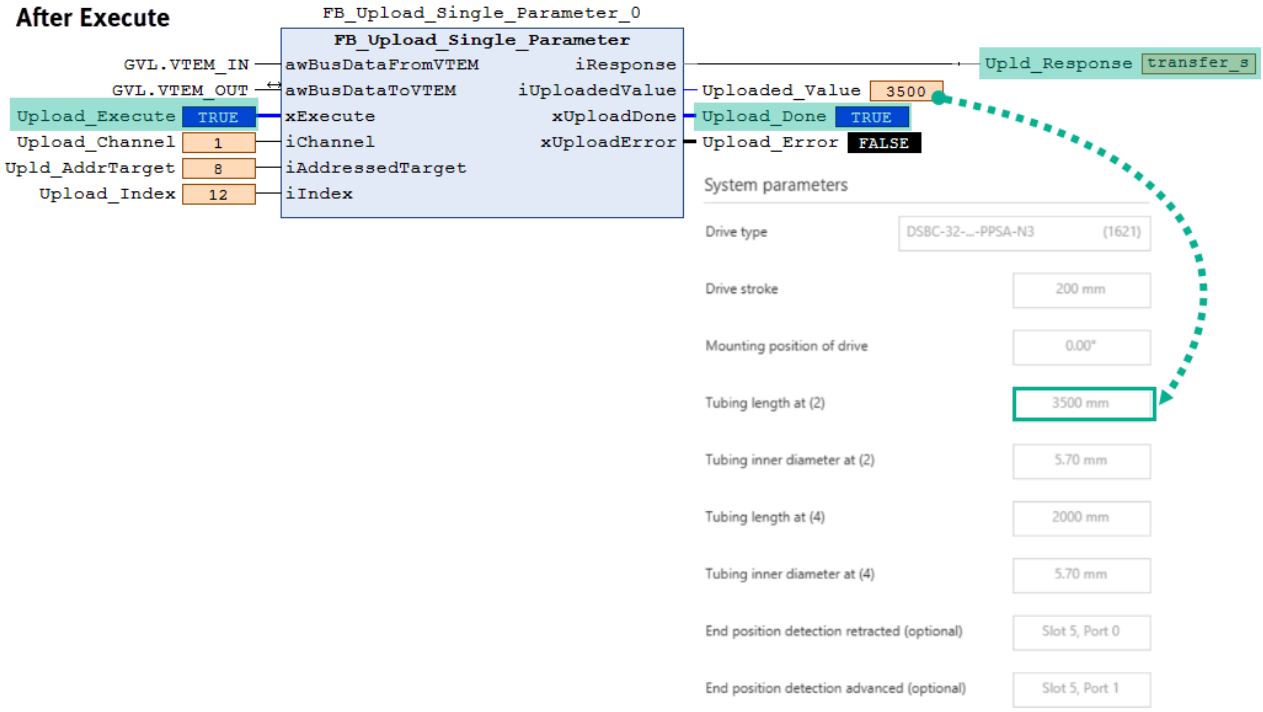
6.3 Sample Code for Uploading Parameter from the VTEM Motion Terminal Valve using FB_Upload_Single_Parameter

This sample code is used to Upload the valve parameter of the VTEM Motion terminal using the Upload single parameter function block.

Before Execute



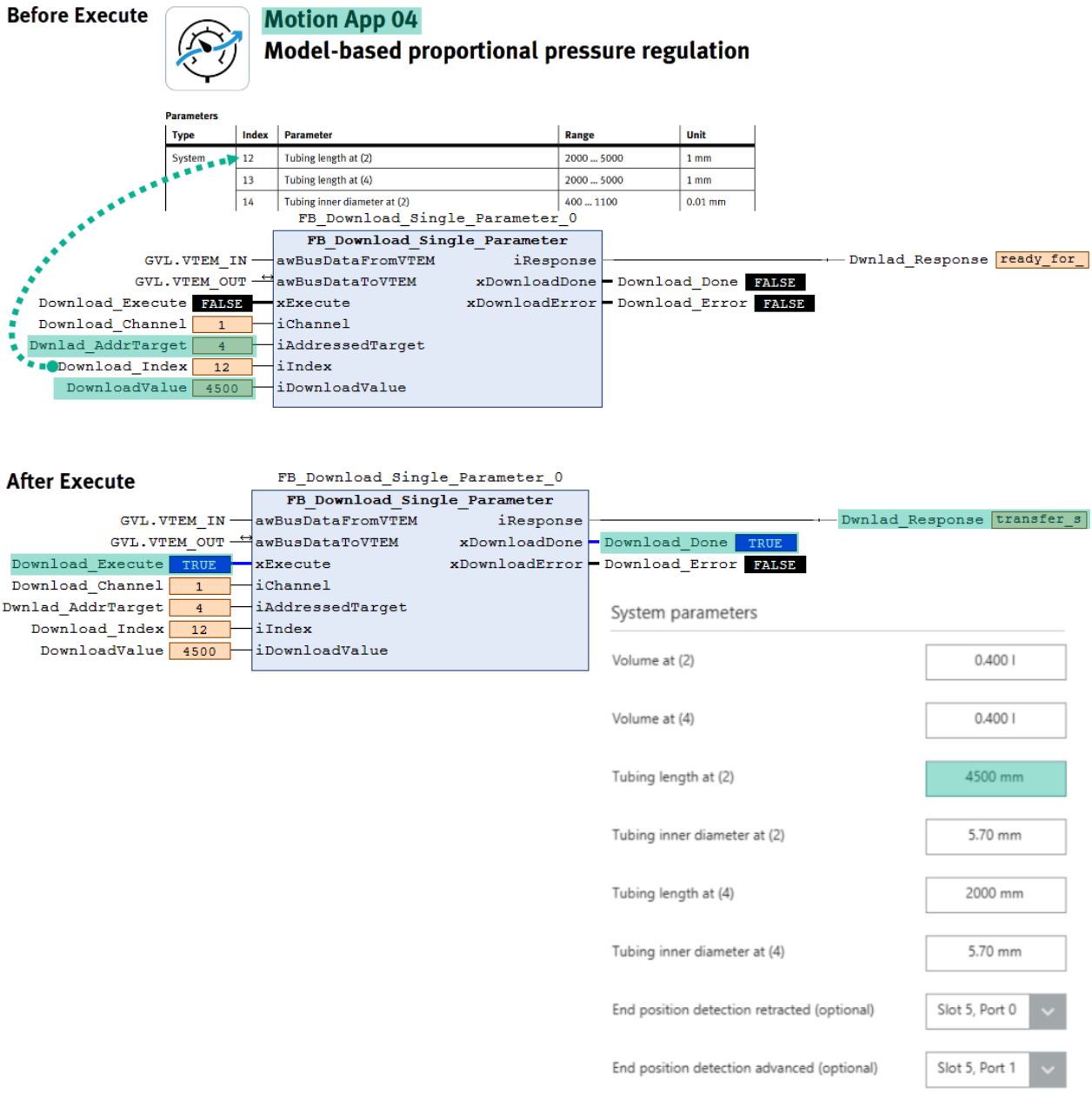
After Execute



1. Enter the channel number, which is the Program SET number, here, **1** (Refer Chapter 6.2) in the **iChannel** input tag.
2. Enter the motion app number in the **iAddressedTarget** input tag.
3. Enter the index number in the **iIndex** input tag.
4. Once all input tags are set, toggle the **xExecute** tag to start the upload process.
5. When the upload succeeds, the monitor tag **xUploadDone** becomes true.
6. The uploaded data is available in the **iUploadedValue** monitor tag .
7. The response of the upload process is available in the **iResponse** monitor tag. (Refer Chapter 4.7.4)
8. If an error occurs, the **xUploadError** monitor tag indicates it.

6.4 Sample Code for Downloading Parameter to the VTEM Motion Terminal Valve using FB_Download_Single_Parameter

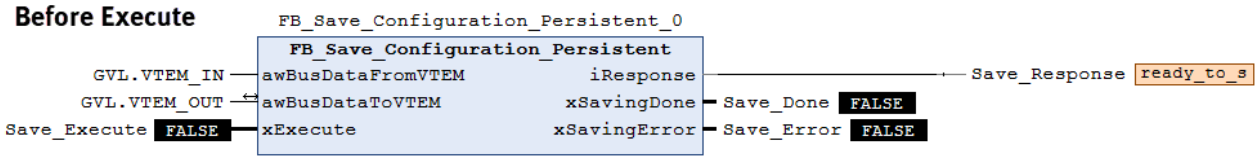
This sample code is used to Download the valve parameter of the VTEM Motion terminal using the Download single parameter function block.

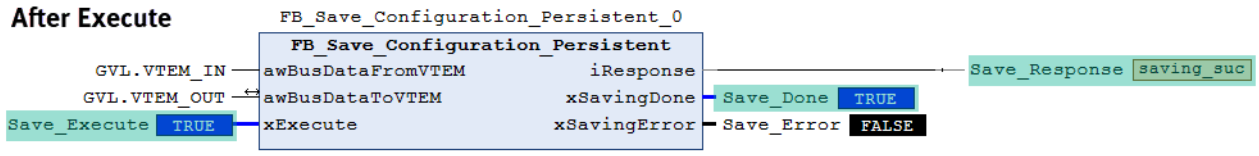


1. Enter the channel number, which is the Program SET number, here, **1** (Refer Chapter 6.2) in the **iChannel** input tag.
2. Enter the motion app number in the **iAddressedTarget** input tag.
3. Enter the index number in the **iIndex** input tag.
4. Enter the parameter value in the **iDownloadValue** input tag.
5. Once all input tags are set, toggle the **xExecute** tag to start the download process.
6. When the download succeeds, the monitor tag **xDownloadDone** becomes true.
7. User will get the response of download process in **iResponse** monitor tag in Integer format. (Refer Chapter 4.7.4)
8. If an error occurs, the **xDownloadError** monitor tag indicates it.

6.5 Sample Code for Save the VTEM Motion Terminal Valve Parameter Using FB_Save_Configuration

This sample code is used to Save the valve parameter of the VTEM Motion terminal using the Save configuration function block.

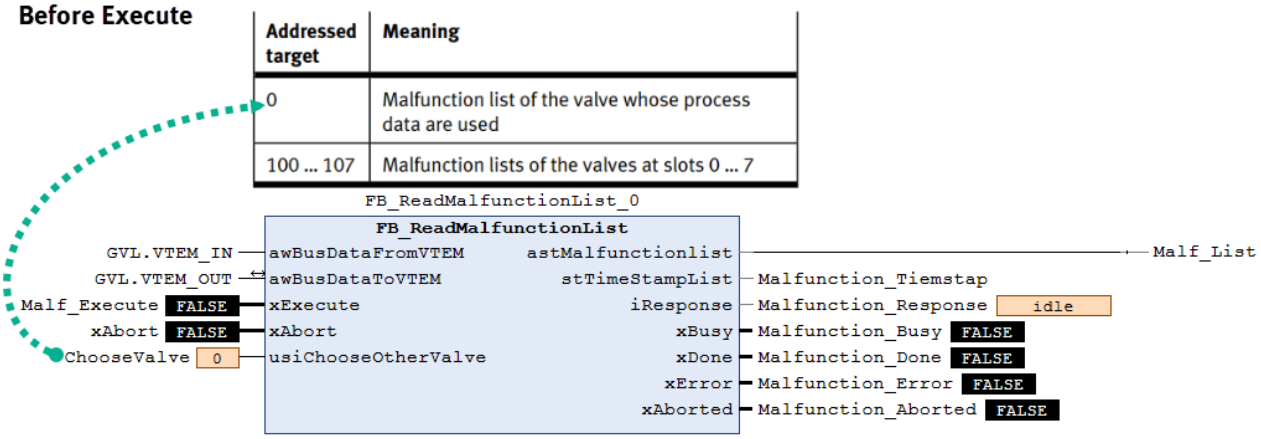




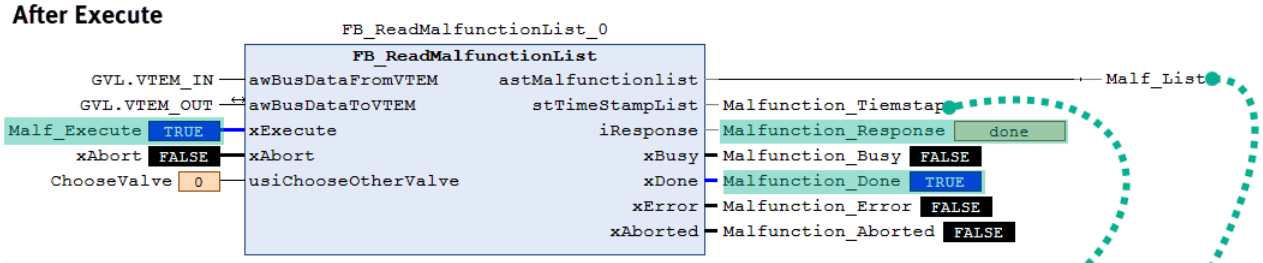
1. Toggle the **xExecute** tag to start the save process.
2. When the save completes successfully, the **xSavingDone** monitor tag becomes TRUE.
3. The response of the save process is available in the **iResponse** monitor tag.(Refer Chapter 4.7.3)
4. If an error occurs, the **xSavingError** monitor tag indicates the error.

6.6 Sample Code for Read the Malfunction list of the VTEM Motion Terminal Valve Using FB_ReadMalfunctionList

This sample code is used to read the malfunction list of the VTEM Motion terminal using the Read Malfunction list function block.



After Execute



Malfunction_Tiemstap	stTimeStamp				
uiHours	UINT	1			
usiMinutes	USINT	55			
usiSeconds	USINT	41			
uiMilliseconds	UINT	453			
Malf_List	ARRAY [1..40] ...				
Malf_List[1]	Festo_VTEM.st...				
usiMalfunctionCode	USINT	22			Malfunction Code
usiMalfunctionSubcode	USINT	1			Malfunction Subcode
usiMalfunctionClassification	USINT	2			Malfunction Classification
uiHours	UINT	1			Time stamp (hour...f the malfuncti...
usiMinutes	USINT	49			Time stamp (min...) of the malfun...
usiSeconds	USINT	32			Time stamp (seco...) of the malfun...
uiMilliseconds	UINT	73			Time stamp (milli...nds) of the mal...
Malf_List[2]	Festo_VTEM.st...				
usiMalfunctionCode	USINT	22			Malfunction Code
usiMalfunctionSubcode	USINT	1			Malfunction Subcode
usiMalfunctionClassification	USINT	2			Malfunction Classification
uiHours	UINT	1			Time stamp (hour...f the malfuncti...
usiMinutes	USINT	49			Time stamp (min...) of the malfun...
usiSeconds	USINT	20			Time stamp (seco...) of the malfun...
uiMilliseconds	UINT	714			Time stamp (milli...nds) of the mal...

1. Enter 0 in the **iChooseOtherValve** input tag to request the malfunction list for the valve associated with the current process data, or
2. Enter a value from 100 to 107 in the **iChooseOtherValve** input tag to request the malfunction list for the valve in the corresponding slot (100 → slot 0, 101 → slot 1, …, 107 → slot 7).
3. Toggle the **xExecute** control tag from false to true to read the malfunction list.
4. If **xError** is true, execute the abort function by setting the **xAbort** control tag.
5. After the abort completes, the **xAborted** monitor tag becomes true. Then execute the block again.
6. When the malfunction list is read successfully, the **xDone** monitor tag becomes true.
7. The malfunction list is available in the **stMalfunctionList** structure data tag.
8. The current timestamp is available in the **stTimeStamp** structure data tag.